

Introduction to Computer Systems Design

CS3031 · IIT Madras · Week 1

1 0 1 1 0 0 1 1

Detailed Study Notes

Foundations of the Digital World

- ▶ Course orientation & roadmap
- ▶ Module 1: computing system blocks, the transformation hierarchy, VLSI & Moore's law
- ▶ Representing information with voltages
- ▶ Noise, the digital abstraction, thresholds & noise margins
- ▶ Bits, 2^n , positional notation
- ▶ Boole, Shannon & the birth of digital circuits
- ▶ 31 numericals, 22 subjective Qs, 40 MCQs

Computational Problem
Algorithms
Programming Languages
System Software
HW/SW Interface
Micro-Architecture
Logic Circuits
Devices
Electrons
Quantum Mechanics

Based on lecture slides by Ayon Chakraborty, Department of CSE, IIT Madras. Boxes marked *Beyond the Slides* add context that is not on the slides but helps understanding.

Contents

How to Use These Notes	4
I Course Orientation	6
1 What This Course Is About	7
1.1 Welcome to CS3031	7
1.2 Examples of Modern Computing Systems	7
1.3 Evolution of the Cell Phone Handset	8
1.4 “You can change the world only if you understand it well enough”	9
1.5 Logistics: Textbooks	9
1.6 Learning Outcomes	9
1.6.1 Not “just hardware”	9
1.6.2 Concrete outcomes	10
1.6.3 Plan of attack	10
1.7 Lesson Plan: the Four Modules	10
II Module 1 — Foundations	12
2 Computing System Blocks and Why We Compute	13
2.1 The Three Blocks of Every Computing System	13
2.1.1 What each block does	13
2.1.2 Why split memory and storage? A trade-off	14
2.2 Why Do We Have Computers?	15
2.2.1 The idealised problem: a closed-form answer exists	15
2.2.2 The real problem: no formula, so compute step by step	15
2.3 How Does a Computer Solve Problems? By Orchestrating Electrons!	16
3 The Transformation Hierarchy and Building Complex Systems	17
3.1 The Transformation Hierarchy	17
3.1.1 Walking down the stack with one example	17
3.2 Building Complex Computer Systems: the Physical Hierarchy	18
3.2.1 What each level is	19
3.2.2 How many transistors? Multiply down the hierarchy	19
3.2.3 The MOSFET: the switch at the bottom	20
3.3 Managing Billions of Transistors: VLSI	21

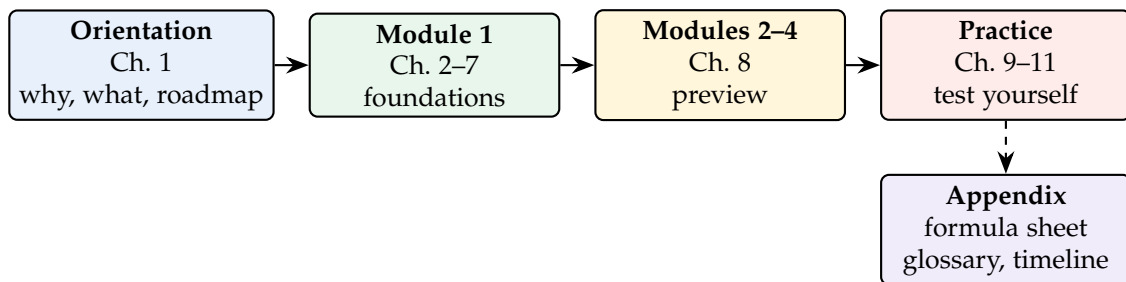
4	Moore's Law and the History of Shrinking Chips	22
4.1	Moore's Law	22
4.1.1	Writing Moore's law as a formula	22
4.2	History: Shrinking Chips	23
4.2.1	Timeline of the milestones	24
5	Representing Information	26
5.1	How Is Information Represented?	26
5.1.1	A bit must live in something physical	26
5.2	Using Electrical Phenomena	27
5.2.1	Why voltage, and what it costs	28
5.3	Representing Information with Voltage: an Image	29
5.3.1	How much information is stored at each point?	29
5.4	Using Voltages to Encode a Picture	30
5.4.1	How the encoding works	31
6	Computation, Noise, and the Digital Abstraction	32
6.1	Information Processing = Computation	32
6.1.1	The functions, precisely	32
6.2	How About This System? Noise Accumulation	33
6.2.1	Why the picture degrades	33
6.2.2	Quantifying accumulation	33
6.3	The Digital Abstraction	34
6.3.1	What the abstraction actually says	35
6.4	Using Voltages "Digitally"	36
6.4.1	What is wrong with a single threshold?	36
6.4.2	How the forbidden zone fixes it	36
6.4.3	Beyond the slides: noise margins and the static discipline	37
6.5	Binary Digital System	38
6.5.1	Why binary rather than 3, 10 or 100 levels?	39
7	Bits, Number Representation, and Boolean Logic	40
7.1	Can We Represent More Than 2 Values?	40
7.1.1	Why n bits give 2^n values	40
7.1.2	The inverse question: how many bits do I need?	40
7.2	Positional Number Representation	41
7.3	Digital Logic $\rightarrow$ Digital Circuits	42
7.3.1	Boolean algebra: logic as arithmetic on $\{0,1\}$	42
7.3.2	Shannon: switches compute Boolean functions	43
7.4	Module 1 Summary: the Whole Week in One Chain of Reasoning	44

III Looking Ahead	45
8 Preview: Where Week 1 Leads (Modules 2–4)	46
8.1 The path from a bit to a CPU	46
8.2 Week 1 idea → later use	46
IV Practice	47
9 Numerical Problems with Worked Solutions	48
9.1 Bits and counting	48
9.2 Positional number representation	49
9.3 Representing information	50
9.4 Noise and the digital abstraction	51
9.5 Hierarchy and Moore’s law	53
9.6 Why we compute	54
10 Subjective Questions with Answers	55
11 Multiple-Choice Questions	60
11.1 Questions	60
11.2 Answers and Explanations	66
A Formula Sheet, Glossary and Quick Revision	68
A.1 Formula sheet	68
A.2 Glossary	69
A.3 One-page revision checklist	69

How to Use These Notes

These notes expand every slide of the Week 1 deck of CS3031 into a self-contained explanation. The slides are short on words by design (they back a live lecture); the goal here is to fill in the *spoken* part: why each idea exists, how it connects to the next one, and what it looks like in numbers.

Structure



The slides' own lesson plan splits the course into four modules. Week 1 belongs entirely to **Module 1 (Foundations, Digital World, Logic Gates, Boolean Algebra, Computer Arithmetic)**, so Chapters 2–7 are the Module 1 content in slide order. Chapter 8 gives a short preview of Modules 2–4 so you can see where each Week 1 idea gets used later.

The coloured boxes

Intuition

The mental picture behind an idea. Read these first if a section feels abstract.

Key Idea

The one sentence you should be able to say from memory.

Formula

A result, with its derivation below it and units shown in grey brackets like [volts] or [bits].

Common Pitfall

A mistake students commonly make in exams.

Beyond the Slides

Material that is *not* on the slides but is standard textbook background (Mano; Harris & Harris) and makes the slide content easier to understand. Useful for later modules; your instructor decides what is examinable.

Notation and units

Symbol	Meaning	Unit
V	voltage	volts (V)
ΔV	smallest reliably distinguishable voltage step	V
n	number of bits	bits
N	number of distinct values / states / levels	dimensionless (count)
$\log_2 N$	information content of choosing one of N equally likely values	bits
T_d	doubling time (Moore's law)	years
V_{IL}, V_{IH}	input low/high thresholds	V
V_{OL}, V_{OH}	output low/high levels	V
NM_L, NM_H	low/high noise margins	V

Indian numbering (used on one slide): 1 lakh = 10^5 , 10 lakh = 10^6 (1 million), 1 crore = 10^7 .

Part I

Course Orientation

Chapter 1

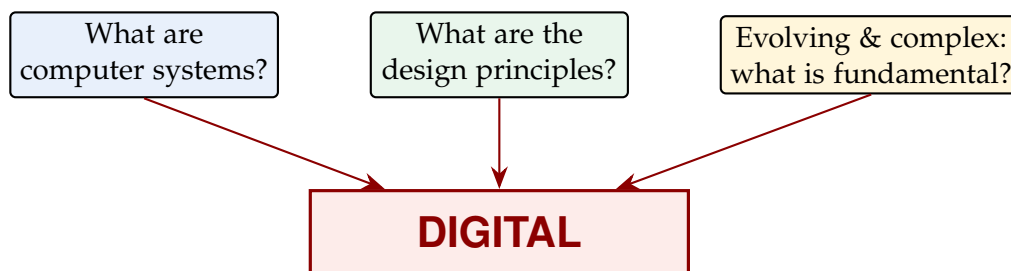
What This Course Is About

1.1 Welcome to CS3031

The opening slide asks three questions that set the agenda for the whole course:

1. What are computer systems?
2. What design principles govern them?
3. They are **evolving and complex** — so what are the **fundamentals** that stay constant?

The slide also flashes one word in large type: **DIGITAL**. That word is the answer to question 3 and the thread through all of Week 1. By the end of these notes you should be able to explain *why* almost every computing system in existence is digital, even though the physical world and the physical components are not.

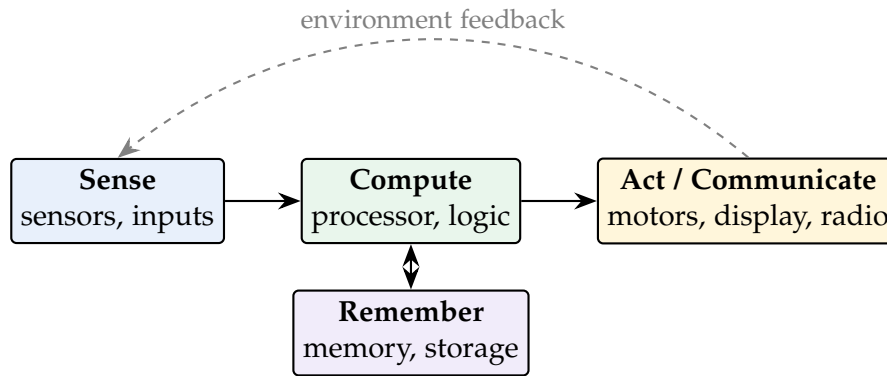


1.2 Examples of Modern Computing Systems

The slide shows a collage: a smart thermostat, a self-driving car with a roof sensor pod, a camera drone, a washing machine, a four-legged robot, a smart speaker, a smartphone, a smartwatch, and smart glasses.

Intuition : the collage is the point

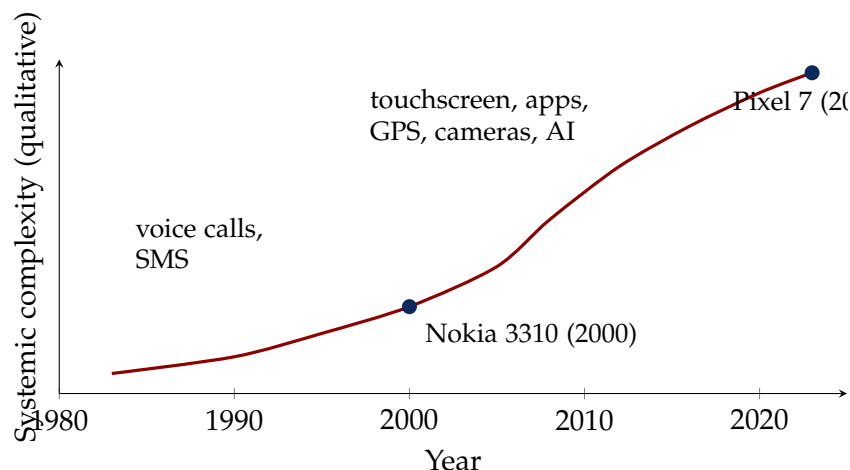
None of these look like the “computer” on your desk, and that is deliberate. A computing system is anything that *senses* something, *computes* a decision from it, and *acts* or *communicates*. The washing machine reads a water-level sensor and runs a timed program; the self-driving car fuses camera, radar and LIDAR data and steers. Same skeleton, very different scale.



System	Senses	Computes	Acts
Smart thermostat	room temperature	compare with set-point, learn schedule	switch heating on/off
Self-driving car	cameras, radar, LIDAR	object detection, path planning	steering, brakes
Drone	IMU, GPS, camera	flight stabilisation loop	motor speeds
Washing machine	water level, door lock	fixed wash program (timers)	valves, drum motor
Legged robot	joint angles, cameras	balance & gait control	joint motors
Smart speaker	microphones	wake-word detection, networking	speaker, radio
Phone / watch / glasses	touch, camera, heart-rate	apps, OS	display, radio

1.3 Evolution of the Cell Phone Handset

The slide plots phones from the 1980s (brick-sized handsets) to the 2020s (all-screen smartphones), with a labelled “Systemic Complexity” axis, contrasting the **Nokia 3310 (2000)** with the **Google Pixel 7 (2023)**.



Intuition : “systemic” complexity

A Nokia 3310 made calls, sent texts, and played Snake. A Pixel 7 has multiple cameras with on-device image processing, a multi-core application processor, a GPU, a machine-learning accelerator, 5G/Wi-Fi/Bluetooth/NFC radios, GPS, and a full operating system. The complexity is *systemic*: not just more transistors, but more interacting subsystems (hardware and software) that must cooperate. The course question is: *how can anyone design something like*

that? The answer, developed below, is **layers of abstraction** built on a **digital** foundation.

1.4 “You can change the world only if you understand it well enough”

The slide lists what “understanding well enough” means:

- the **basics** (fundamentals);
- the **past and present dominant paradigms**;
- their **advantages and shortcomings — trade-offs**;
- **what remains fundamental across generations**;
- the **techniques** you can use and develop to solve design problems.

Key Idea

Technologies change every few years (vacuum tubes → transistors → integrated circuits → today’s chips); *principles* do not. Binary representation, Boolean logic, abstraction layers and the stored-program organisation have survived every generation. Engineering is choosing between trade-offs (speed vs power vs cost vs reliability), and you can only reason about trade-offs if you know the fundamentals.

1.5 Logistics: Textbooks

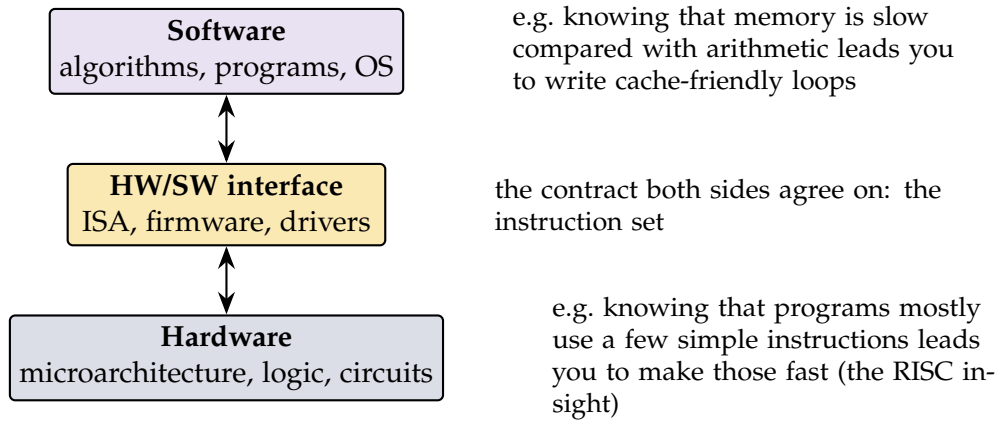
Role	Book	Why it is useful
Main text	M. Morris Mano, <i>Digital Logic and Computer Design</i> (Pearson)	Classic, bottom-up: number systems, Boolean algebra, combinational & sequential logic, then processor design. Matches Modules 1–3.
Advanced reference	Sarah L. Harris & David Money Harris, <i>Digital Design and Computer Architecture, ARM Edition</i> (Morgan Kaufmann)	Modern treatment, goes further into microarchitecture and a real ISA (ARM). Matches Module 4 and beyond. Its Chapter 1 covers exactly the Week 1 ideas (digital abstraction, noise margins).

1.6 Learning Outcomes

1.6.1 Not “just hardware”

The slides stress that although the course looks like “computer hardware”, you become much more capable if you master **both hardware and software and the interface between them**:

- You can **develop better software** if you understand the hardware.
- You can **design better hardware** if you understand the software.
- You can **design a better computing system** if you understand both.



Later courses that build on this one (named on the slide): Computer Organization, Computer Architecture, Operating Systems, Compilers, Networks, Embedded Systems/IoT, VLSI. In them you will learn more about the HW/SW interface and microarchitecture, and keep meeting **trade-offs** and how they affect system software.

1.6.2 Concrete outcomes

1. You will be able to **conceptualise and design simple digital logic blocks** that act as the building blocks of a computing system.
2. You will be able to **design a simple central processing unit (CPU)**: an Arithmetic and Logic Unit (ALU) plus the **timing and control units**.

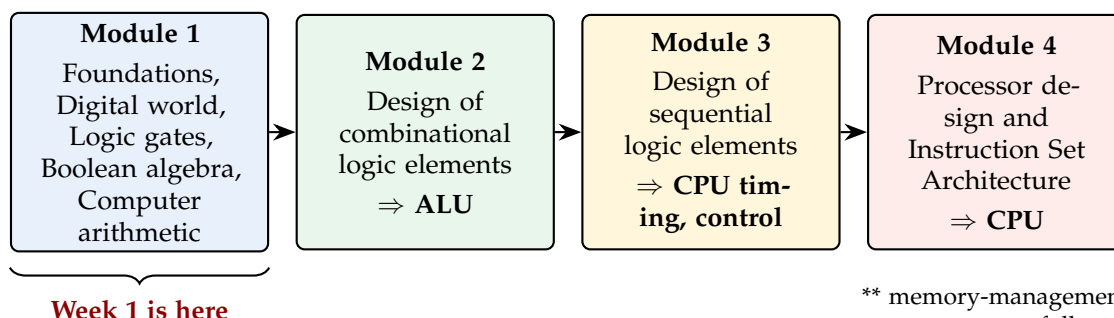
1.6.3 Plan of attack

- Understand how things work **bottom-up**.
- **Encapsulate** that understanding in appropriate **abstractions**, so the next level up does not need to re-think the details.

Intuition : bottom-up, then seal it

Learn how a transistor switches; seal it into a *gate*. Learn how gates combine; seal them into an *adder*. Adders and multiplexers become an *ALU*; ALU + registers + control become a *CPU*. At each step you stop caring about the inside of the box you just built and only use its interface. This is exactly how real billion-transistor chips are designed.

1.7 Lesson Plan: the Four Modules



** memory-management concepts may follow, depending on progress

Key Idea

Module 1 gives you the *language* (bits, Boolean algebra, number representation). Module 2 uses it to build circuits *without memory* (output depends only on current inputs: adders, multiplexers → ALU). Module 3 adds *memory and time* (flip-flops, registers, counters, state machines → timing and control). Module 4 assembles everything into a processor that executes instructions.

Part II

Module 1 — Foundations

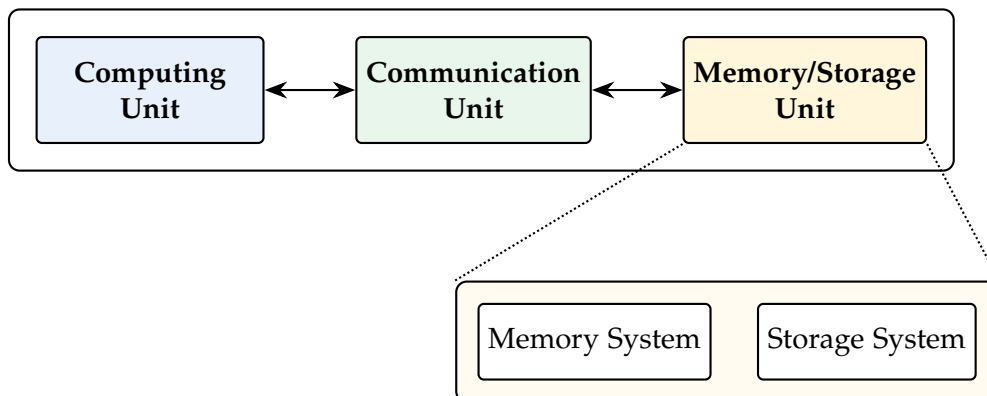
Chapter 2

Computing System Blocks and Why We Compute

2.1 The Three Blocks of Every Computing System

From the slides:

Computing System Blocks A box diagram: **Computing Unit** ↔ **Communication Unit** ↔ **Memory/Storage Unit**, where the Memory/Storage Unit is expanded into a **Memory System** and a **Storage System**. Reference: Burks, Goldstein, von Neumann, “*Preliminary discussion of the logical design of an electronic computing instrument*”, 1946. The photo shows John von Neumann standing beside an early electronic computer.



2.1.1 What each block does

Computing unit Transforms data: arithmetic (+, −, ×), logic (AND, OR, NOT, comparisons) and the *control* that decides which operation happens next. In a CPU this is the ALU plus control unit (Modules 2–4).

Communication unit Moves data between places: on-chip wires and buses, the interconnect between processor and memory, and external links (network, USB, radios). Every datum that is computed on must first be *brought* to the computing unit.

Memory/Storage unit Holds data and programs over time.

- **Memory system:** fast, relatively small, usually *volatile* (contents lost on power-off). Registers, caches, main memory (DRAM).
- **Storage system:** slower, much larger, *non-volatile*. SSDs, hard disks, flash cards.

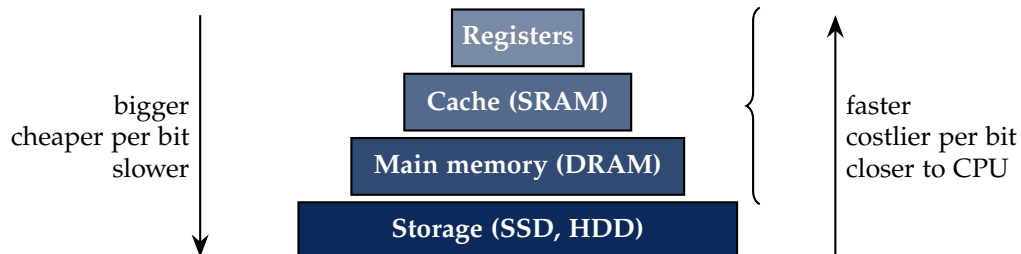
Intuition : a kitchen

The chef is the computing unit; the counter-top is memory (small, right at hand, cleared at closing time); the pantry and freezer are storage (big, slow to walk to, keeps things overnight); the waiter and the corridors are the communication unit. A great chef is useless if the ingredients arrive slowly — in modern computers *moving* data often costs more time and

energy than *computing* on it.

2.1.2 Why split memory and storage? A trade-off

No single technology is simultaneously fast, large, cheap and non-volatile. So systems use a *hierarchy*:

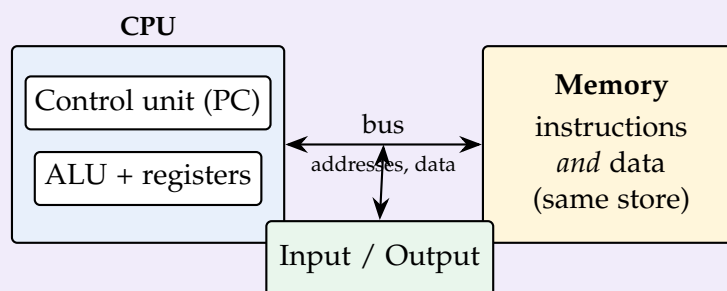


The top three levels form the “memory system” of the slide; the bottom level is the “storage system”.

Beyond the Slides : the von Neumann (stored-program) architecture

The 1946 Burks–Goldstein–von Neumann report proposed the organisation that almost every computer still follows:

1. **Stored program**: instructions are stored in the *same* memory as data, as numbers. A program can therefore be loaded, changed, or even generated by another program, without rewiring the machine (earlier machines such as ENIAC were largely programmed by plugging cables).
2. **Binary** representation for both data and instructions.
3. A processor that repeatedly **fetches** an instruction from memory, **decodes** it, and **executes** it, with a *program counter* pointing to the next instruction.



Because instructions and data share one path to memory, the processor can only move a limited amount per unit time. This limit is called the **von Neumann bottleneck**, and a large part of computer architecture (caches, wider buses, prefetching) exists to work around it. You will see this again in Module 4.

2.2 Why Do We Have Computers?

From the slides:

Why Do We Need Computation? **To solve problems** — to gain and generate insights by solving problems. (Illustrated with a cannon firing a ball along a curved trajectory.)

The cannon is not a random picture. Computing *firing tables* (where will a shell land for a given elevation and charge?) was one of the original motivations for building ENIAC during the Second World War. It is a perfect example of a problem that is easy to *state* and tedious to *solve*.

2.2.1 The idealised problem: a closed-form answer exists

Launch a projectile at speed v_0 [m/s] and angle θ [rad] with gravity g [m/s²] and no air resistance.

Horizontal and vertical motion are independent:

$$x(t) = v_0 \cos \theta t \quad [\text{m} = (\text{m/s})(\text{s})] \quad (2.1)$$

$$y(t) = v_0 \sin \theta t - \frac{1}{2}gt^2 \quad [\text{m} = (\text{m/s})(\text{s}) - (\text{m/s}^2)(\text{s}^2)] \quad (2.2)$$

The ball lands when $y(t) = 0$ with $t > 0$:

$$t(v_0 \sin \theta - \frac{1}{2}gt) = 0 \Rightarrow t_{\text{flight}} = \frac{2v_0 \sin \theta}{g} \quad [\text{s} = (\text{m/s})/(\text{m/s}^2)]$$

Substituting into $x(t)$ and using $2 \sin \theta \cos \theta = \sin 2\theta$:

Formula : projectile range (no drag)

$$R = x(t_{\text{flight}}) = v_0 \cos \theta \cdot \frac{2v_0 \sin \theta}{g} = \frac{v_0^2 \sin 2\theta}{g} \quad [\text{m} = (\text{m}^2/\text{s}^2)/(\text{m/s}^2)]$$

Maximum range at $\theta = 45^\circ$: $R_{\text{max}} = v_0^2/g$.

2.2.2 The real problem: no formula, so compute step by step

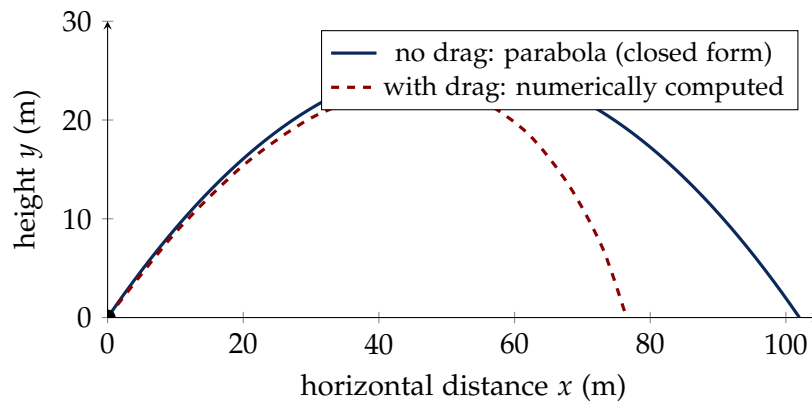
Real shells feel air drag roughly proportional to v^2 , and air density changes with height. The equations become

$$\ddot{x} = -\frac{k}{m} v \dot{x}, \quad \ddot{y} = -g - \frac{k}{m} v \dot{y}, \quad v = \sqrt{\dot{x}^2 + \dot{y}^2},$$

where k [kg/m] is a drag constant and m [kg] the mass (check: $\frac{k}{m}v\dot{x}$ has units $\frac{\text{kg/m}}{\text{kg}} \cdot \frac{\text{m}}{\text{s}} \cdot \frac{\text{m}}{\text{s}} = \text{m/s}^2$, an acceleration, as it must). These have no neat closed-form solution. Instead we *march forward in small time steps* Δt :

$$v_x \leftarrow v_x + a_x \Delta t, \quad v_y \leftarrow v_y + a_y \Delta t, \quad x \leftarrow x + v_x \Delta t, \quad y \leftarrow y + v_y \Delta t.$$

Thousands of multiply-add steps per trajectory, hundreds of trajectories per table: exactly the repetitive, error-prone arithmetic that humans are bad at and machines are good at.



Key Idea

A computer is a machine for turning a *problem* into an *answer* (and, through the answer, into *insight*) by performing huge numbers of simple, precisely defined steps. The rest of the course is about how simple steps are physically carried out.

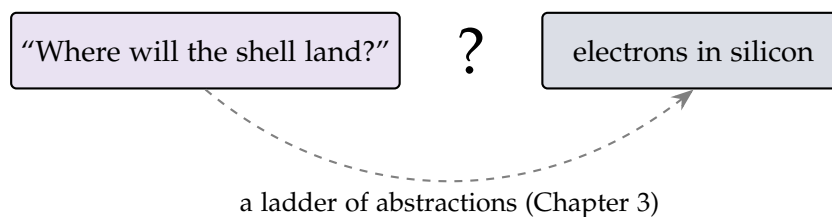
2.3 How Does a Computer Solve Problems? By Orchestrating Electrons!

From the slides:

How Does A Computer Solve Problems? **By orchestrating electrons!** Followed by the question: *How do problems get solved by electrons?*

At the bottom of every computation is physics: electrons pushed through silicon by electric fields. A transistor lets a small voltage on one terminal control whether current flows between two others. Arrange billions of these switches correctly and the pattern of which ones are on and off *is* the computation.

But a gap separates “I want to compute a firing table” from “move these electrons”. Nobody bridges it in one leap. The bridge is a stack of **abstraction layers**, each built on the one below — the *transformation hierarchy*, next chapter.

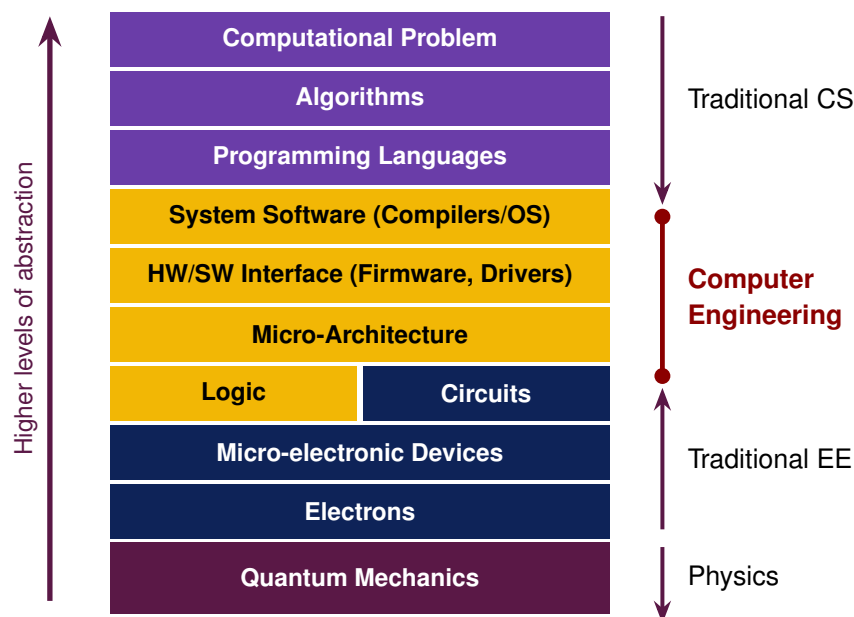


The Transformation Hierarchy and Building Complex Systems

3.1 The Transformation Hierarchy

From the slides:

The Transformation Hierarchy A stack, from top to bottom: Computational Problem; Algorithms; Programming Languages; System Software (Compilers/OS); HW/SW Interface (Firmware, Drivers); Micro-Architecture; Logic | Circuits; Micro-electronic Devices; Electrons; Quantum Mechanics. An upward arrow on the left reads “Higher levels of abstraction”. On the right: *Traditional CS* covers the top down to system software; *Traditional EE* covers electrons/devices/circuits upward; *Computer Engineering* is the bridge in the middle; *Physics* is below.



3.1.1 Walking down the stack with one example

Take the problem “sort a list of exam marks”.

Level	What the “sort marks” problem looks like here
Computational problem	Given n numbers, output them in non-decreasing order.
Algorithm	Merge sort: split, sort halves, merge. $O(n \log n)$ comparisons.
Programming language	<code>sorted(marks)</code> in Python, or a C function with loops and arrays.
System software	The compiler turns C into machine instructions; the OS loads the program, gives it memory, schedules it on a core.
HW/SW interface	The instruction set: <code>LOAD, CMP, BRANCH, STORE ...</code> ; firmware and drivers let the OS talk to devices.
Micro-architecture	Pipeline stages, register file, ALU, caches that actually carry out each instruction.
Logic	Comparators, adders, multiplexers built from AND/OR/NOT gates.
Circuits	Each gate as a network of transistors (e.g. CMOS).
Micro-electronic devices	MOSFETs: voltage on the gate controls a channel between source and drain.
Electrons	Charge carriers flowing (or not) through doped silicon.
Quantum mechanics	Band structure of silicon explains why doping and field effects work at all.

Intuition : abstraction = a contract

Each layer offers the layer above a simplified *contract* and hides how it keeps that contract. A Python programmer does not think about electrons; a circuit designer does not think about merge sort. This is the only reason a single human can contribute to a system with billions of parts: they work at one layer and trust the contracts above and below.

Key Idea

Computer engineering sits in the *middle* of the stack — system software down to logic and circuits — and is the bridge between what software needs and what physics can provide. This course lives mostly in **Logic** and **Micro-architecture**, with glances up to the HW/SW interface.

Common Pitfall

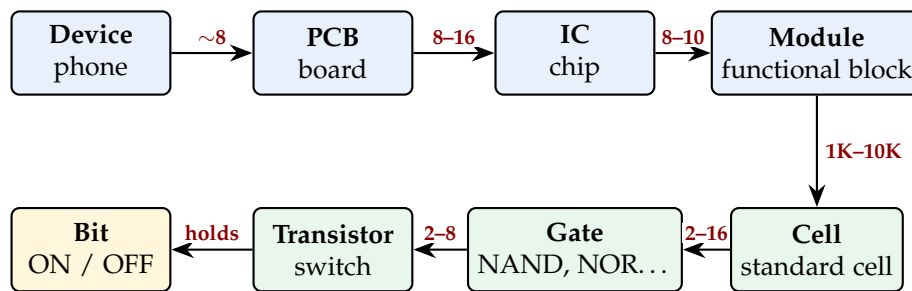
Abstractions *leak*. A program that walks memory in a cache-unfriendly order can run many times slower than an equivalent one, even though the programming-language level says they are “the same algorithm”. Good engineers know at least one layer above and below their own — which is exactly the slide’s HW/SW message.

3.2 Building Complex Computer Systems: the Physical Hierarchy

The transformation hierarchy is about *concepts*. This slide shows the same idea *physically*: zoom into a phone, step by step, until you reach a single transistor.

From the slides:

Building Complex Computer Systems Computer (Google Pixel XL) → PCB (~8 per device) → IC (8–16 per PCB) → Module (8–10 per IC) → Cell (1K–10K per module) → Gate (2–16 per cell) → Transistor (2–8 per gate). A MOSFET cross-section shows Source, Gate (gate metallisation over an oxide layer), Drain, N^+ regions in P-silicon. Bottom-left: 01011001 — *Information: ON/OFF switching*.



numbers on arrows = how many of the next level are inside one of the current level

3.2.1 What each level is

PCB (printed circuit board) Fibreglass board with copper traces that connect chips, connectors, and passive components. A phone has a main logic board plus smaller boards (camera, charging port, etc.).

IC (integrated circuit) A single silicon die in a package: the application processor (SoC), memory chips, power-management IC, radio chips.

Module A functional block *on* the die: a CPU core, a GPU, a cache, a memory controller, an image signal processor. The slide's die photo shows these as distinct rectangular regions.

Cell A pre-designed, pre-verified layout of a small function (an inverter, a NAND, a flip-flop, a full adder) from a *standard-cell library*. Chip designers tile thousands of identical-height cells in rows. The coloured layout on the slide (metal layers, polysilicon, diffusion) is one such cell.

Gate A logic function: NOT, AND, OR, NAND, NOR, XOR.

Transistor The switch. On a digital chip: a MOSFET.

3.2.2 How many transistors? Multiply down the hierarchy

Formula : counting transistors through a hierarchy

If each level contains m_i units of the level below, the total count of the bottom unit is the product

$$N_{\text{transistors}} = \prod_i m_i \quad [\text{count} = \text{count} \times \text{count} \times \dots]$$

Derivation. One device holds m_1 PCBs; each PCB holds m_2 ICs, so there are $m_1 m_2$ ICs. Each IC holds m_3 modules, so $m_1 m_2 m_3$ modules, and so on down. Every level multiplies.

Using the slide's ranges, **per IC** (IC $\rightarrow$ transistor):

$$N_{\min}^{\text{IC}} = 8 \times 1000 \times 2 \times 2 = 3.2 \times 10^4,$$

$$N_{\max}^{\text{IC}} = 10 \times 10\,000 \times 16 \times 8 = 1.28 \times 10^7.$$

Per device (multiply by ICs/PCB and PCBs/device):

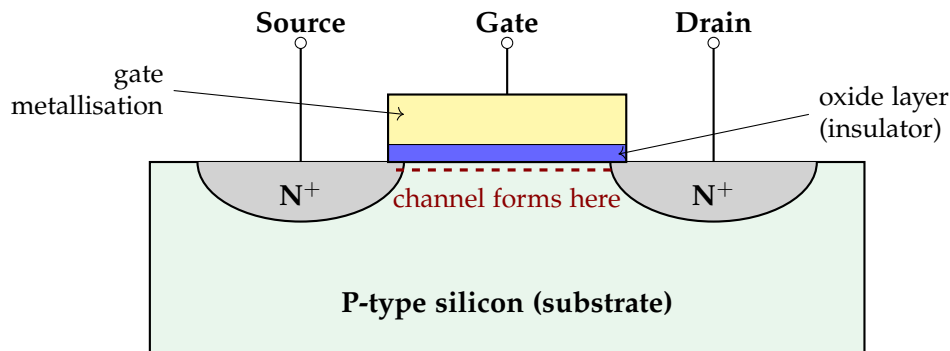
$$N_{\min}^{\text{dev}} = 8 \times 8 \times 3.2 \times 10^4 \approx 2.0 \times 10^6,$$

$$N_{\max}^{\text{dev}} = 8 \times 16 \times 1.28 \times 10^7 \approx 1.6 \times 10^9.$$

So even the slide's modest, round-number multipliers take you from a handful of switches to **billions**. (Real modern SoCs are larger still: tens of billions of transistors on a single die. The slide's multipliers are illustrative, not exact for any specific phone.)

Intuition : why a hierarchy is unavoidable

Nobody can reason about 10^9 things individually. But everybody can reason about ~ 10 things. A hierarchy with $\sim 6-8$ levels, each of fan-out 10–1000, lets humans (and design tools) handle 10^9 by only ever looking at one level at a time. The number of levels grows like $\log(\text{total})$, which is small.

3.2.3 The MOSFET: the switch at the bottom

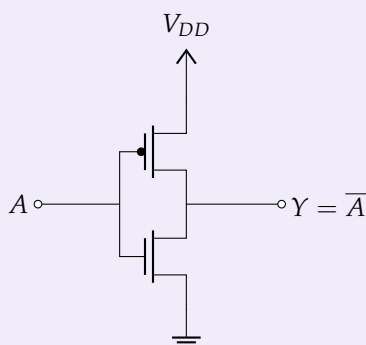
- Source and drain are heavily doped N^+ regions (lots of free electrons) in a **P-type** substrate. Between them, the P-region blocks current: the switch is **OFF**.
- Apply a sufficiently **positive voltage to the gate**. The gate and substrate, separated by a thin **oxide** insulator, act like a capacitor. The electric field pulls electrons to the surface just under the oxide, forming a thin N-type **channel** that connects source to drain: the switch is **ON**.
- No current flows *into* the gate (it is insulated), so in steady state it takes almost no power to hold a switch on or off.

Key Idea

Information in hardware is **ON/OFF switching**. The bit pattern 01011001 on the slide literally corresponds to a row of switches, some conducting and some not. Everything else in the hierarchy is organisation of these switches.

Beyond the Slides : nMOS, pMOS and the CMOS inverter

The device on the slide is an **nMOS** transistor: ON when the gate is high. A **pMOS** transistor is the mirror image (P^+ regions in N-type silicon) and is ON when the gate is *low*. **CMOS** (complementary MOS) pairs them so that for every input one path is on and the other off:



A	pMOS / nMOS	Y
0	ON / OFF	1 (pulled to V_{DD})
1	OFF / ON	0 (pulled to ground)

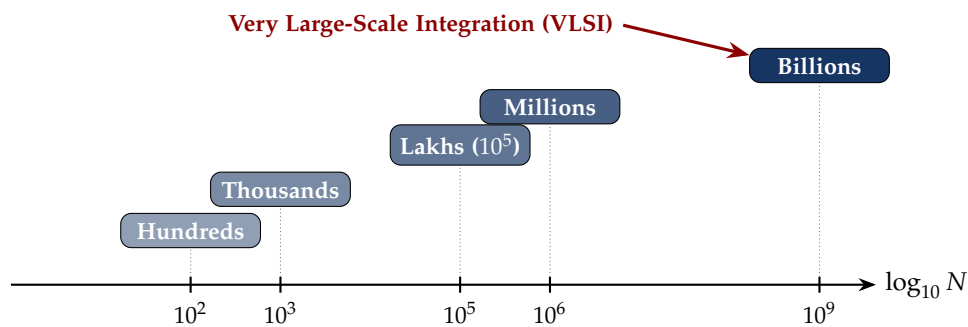
Since there is never a direct path from V_{DD} to ground in steady state, CMOS draws almost no static power. This is the “very low power in steady state” advantage of voltage encoding that appears later in the slides, and it is why the slide’s history lists the **first CMOS gate (1963)**

and **first commercial CMOS chips (1974, digital watches)** — watches run for years on a tiny battery precisely because of this property.

3.3 Managing Billions of Transistors: VLSI

From the slides:

Building Complex Computer Systems (scale) The same hierarchy greyed out, with a ladder of scales: *Hundreds* → *Thousands* → *Lakhs* → *Millions* → **Billions** — “Managing Billions of Transistors!” — pointing to **Very Large-Scale Integration (VLSI)**.



Beyond the Slides : integration scales

Historically, chips were classified by how many components they integrate. The boundaries are loose conventions, roughly:

Scale	Name	Gates / transistors per chip	Era & example
SSI	Small-scale integration	up to ~10s of gates	1960s: a chip with four NAND gates
MSI	Medium-scale	~10s–100s of gates	late 1960s: counters, adders
LSI	Large-scale	~1 000s–10 000s	1970s: early microprocessors (4004)
VLSI	Very large-scale	10^5 upward	1980s onward: all modern processors

Today everything is “VLSI”; the term now names the discipline of designing such chips (which is also the name of a later course).

Key Idea

VLSI is only possible *because of* the hierarchy and the digital abstraction. Designers describe behaviour at a high level (hardware description languages), and tools map it onto standard cells, which were in turn verified at the transistor level once. Nobody draws a billion transistors by hand.

Moore's Law and the History of Shrinking Chips

4.1 Moore's Law

From the slides:

Moore's Law Moore's law is a term used to refer to the observation made by Gordon Moore in 1965 that the number of transistors in a dense integrated circuit (IC) **doubles about every two years**. Source paper: "Cramming more components onto integrated circuits" (4 pages), by Gordon E. Moore, Director, Research and Development Laboratories, Fairchild Semiconductor division of Fairchild Camera and Instrument Corp. Question on the slide: *Company founded by Moore?*

Answer

Intel, co-founded by Gordon Moore and Robert Noyce in 1968. (Before that, Moore was one of the eight engineers who founded Fairchild Semiconductor in 1957 — which is why the 1965 paper lists his Fairchild affiliation.)

Beyond the Slides : "every year" vs "every two years"

Moore's original 1965 paper extrapolated a doubling of components per chip roughly *every year*. In 1975 he revised the rate to roughly *every two years*, which is the form usually quoted today and the form used on the slide. It is an **empirical observation** (and later an industry planning target), not a law of physics.

4.1.1 Writing Moore's law as a formula

"Doubles every T_d years" means: after one period the count is $\times 2$, after two periods $\times 4$, after k periods $\times 2^k$. After $t - t_0$ years there have been $(t - t_0)/T_d$ periods, so:

Formula : Moore's law

$$N(t) = N_0 \cdot 2^{(t-t_0)/T_d} \quad [\text{transistors} = \text{transistors} \times (\text{dimensionless})]$$

N_0 : transistor count in reference year t_0 ; T_d : doubling time [years]; the exponent $(t - t_0)/T_d$ is dimensionless (years/years). With $T_d = 2$ years (slide):

$$N(t) = N_0 \cdot 2^{(t-t_0)/2}.$$

Equivalent forms.

- *Annual growth factor*. Set $t - t_0 = 1$: $N(t_0 + 1)/N_0 = 2^{1/T_d}$. For $T_d = 2$: $2^{0.5} = \sqrt{2} \approx 1.414$, i.e. about **41% more transistors per year**.

- *Exponential (base e) form.* Since $2^x = e^{x \ln 2}$:

$$N(t) = N_0 e^{r(t-t_0)}, \quad r = \frac{\ln 2}{T_d} \quad [1/\text{year}]$$

For $T_d = 2$: $r = 0.3466$ per year.

- *Time for a $10\times$ increase.* Solve $2^{\Delta t/T_d} = 10$: $\Delta t = T_d \log_2 10 \approx 3.32 T_d$. For $T_d = 2$: **≈ 6.6 years per factor of 10**, so **≈ 33 years per factor of 10^5** .
- *Doubling time from two data points.* Given (t_1, N_1) and (t_2, N_2) :

$$\frac{N_2}{N_1} = 2^{(t_2-t_1)/T_d} \Rightarrow T_d = \frac{t_2 - t_1}{\log_2(N_2/N_1)} = \frac{(t_2 - t_1) \ln 2}{\ln(N_2/N_1)} \quad [\text{years}]$$

Worked Example : checking the law against real chips

Intel 4004 (1971): ≈ 2300 transistors. Intel Ivy Bridge (2012): $\approx 1.4 \times 10^9$.

$$\frac{N_2}{N_1} = \frac{1.4 \times 10^9}{2.3 \times 10^3} \approx 6.09 \times 10^5, \quad \log_2(6.09 \times 10^5) = \frac{\ln(6.09 \times 10^5)}{\ln 2} = \frac{13.32}{0.693} \approx 19.2.$$

$$T_d = \frac{2012 - 1971}{19.2} = \frac{41}{19.2} \approx 2.1 \text{ years.}$$

About 19 doublings in 41 years: the “two years” rule held remarkably well for four decades.

Intuition : why a straight line on a log plot

Take $\log_{10}$ of Moore’s law: $\log_{10} N = \log_{10} N_0 + \frac{\log_{10} 2}{T_d}(t - t_0)$. That is a straight line in t with slope $\frac{0.301}{T_d}$ decades per year (0.15 decades per year for $T_d = 2$). So exponential growth looks like a straight line when the y -axis is logarithmic — which is exactly how the next slide plots it.

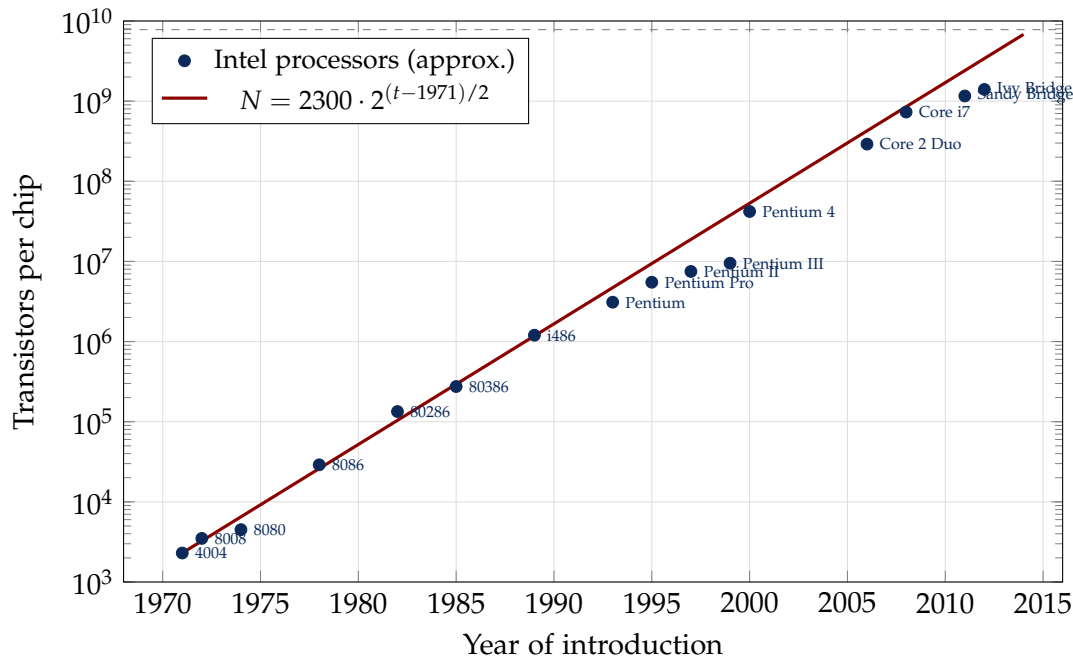
Common Pitfall

Doubling every 2 years is *not* “ $+2\times$ every year” or “ $+100\%$ every year”. It is $\times \sqrt{2} \approx \times 1.41$ per year. And after 10 years it is $2^5 = 32\times$, not $10\times$ or $20\times$.

4.2 History: Shrinking Chips

From the slides:

History: Shrinking Chips Semi-log plot of transistors per chip (from 10^3 to 10^9) vs year of introduction (1970–2020), with a straight trend line through Intel and AMD processors: 4004, 8008, 8080, 8086, 80286, MC68000, 80386, MC68020, i486, i860, Pentium, Pentium Pro, Pentium II, Pentium III, AMD K6, AMD Athlon, Pentium 4, Pentium M, Core Duo, Core2 Duo, Core2 Quad, AMD K8, AMD K10, Core i7, Sandy Bridge, Gulftown Core 6, Ivy Bridge. Milestones listed: first transistor 1947 (Nobel prize); first silicon transistor 1954; first CMOS gate 1963; first commercial CMOS chips 1974 (digital watches); as of 2020 some of the largest chips have more than 10 000 000 000 transistors — about as many as (or more than) people on Earth.



Data points are commonly cited approximate transistor counts; they are for illustration of the trend.

4.2.1 Timeline of the milestones

- 1854 ● George Boole: logic is algebra on binary values
- 1906 ● Lee de Forest: vacuum tube (triode), an electronic switch
- 1937 ● Claude Shannon's MIT thesis: switch circuits obey Boolean algebra
- 1946 ● Burks–Goldstein–von Neumann report: stored-program computer
- 1947 ● First transistor (Bell Labs) — Nobel Prize 1956
- 1954 ● First silicon transistor
- 1963 ● First CMOS gate
- 1965 ● Moore's paper: "Cramming more components onto integrated circuits"
- 1971 ● Intel 4004: $\approx 2,300$ transistors
- 1974 ● First commercial CMOS chips (digital watches)
- 2020 ● Largest chips exceed 10^{10} transistors ($\approx$ world population)

blue: ideas & architecture red: devices & chips (spacing not to scale)

1947 — first transistor. Built at Bell Labs by John Bardeen, Walter Brattain and William Shockley (a point-contact transistor made of germanium). They shared the 1956 Nobel Prize in Physics. It replaced the vacuum tube as a switch/amplifier: smaller, cooler, more reliable.

1954 — first silicon transistor. Silicon became the material of choice: it tolerates higher temperatures than germanium and forms an excellent native oxide (SiO_2), which is exactly the insulating layer in the MOSFET drawing of Chapter 3.

1963 — first CMOS gate. Complementary nMOS/pMOS pairs, drawing almost no power when idle.

1974 — first commercial CMOS chips in digital watches, the killer application for low power.

2020 — more than 10^{10} transistors on the largest chips, comparable to the number of humans on Earth ($\approx 7.8 \times 10^9$ in 2020).

Intuition : what “shrinking” buys you

The chart counts transistors, but the driver was making each transistor *smaller*. If the linear feature size shrinks by a factor s , each transistor’s area shrinks by s^2 , so about s^2 times as many fit on the same area. A doubling of density every two years corresponds to linear shrink by $\sqrt{2} \approx 1.41$ every two years ($s^2 = 2$). Smaller transistors are also faster (shorter distances, smaller capacitances) and, for a long time, used less energy per switch. That combination is why computers got better *and* cheaper at once.

Formula : density vs feature size

If transistor area $A \propto L^2$ where L is the feature size [m], then transistors per unit area

$$D \propto \frac{1}{L^2} \quad [\text{transistors/m}^2 \propto 1/\text{m}^2], \quad \frac{D_2}{D_1} = \left(\frac{L_1}{L_2} \right)^2.$$

Doubling density ($D_2/D_1 = 2$) requires $L_1/L_2 = \sqrt{2}$, i.e. $L_2 \approx 0.707 L_1$ — the classic “0.7× per generation” shrink.

Beyond the Slides : is Moore’s law over?

Growth has slowed. Around the mid-2000s, clock frequencies stopped rising because shrinking transistors no longer reduced power density enough (the end of “Dennard scaling”); chips became hot rather than faster, so the industry turned to *multiple cores* — visible on the slide’s chart as Core Duo, Core2 Quad, Gulftown Core 6. Transistors are now only tens of atoms across in their smallest dimensions, and each new process generation is slower and more expensive to reach. Density still improves, via 3D transistor shapes, stacking dies, and specialised accelerators (GPUs, neural-network engines), but no longer on a clean two-year clock. For this course the lesson is: *when raw transistor growth slows, good architecture — how you organise the transistors — matters more.*

Representing Information

5.1 How Is Information Represented?

From the slides:

Representing Information We understand that information needs to be stored using bits. But where do these bits come from? Computer systems store, retrieve, manipulate (process) such information (bits). How should this information be physically encoded? **What makes a good bit?**

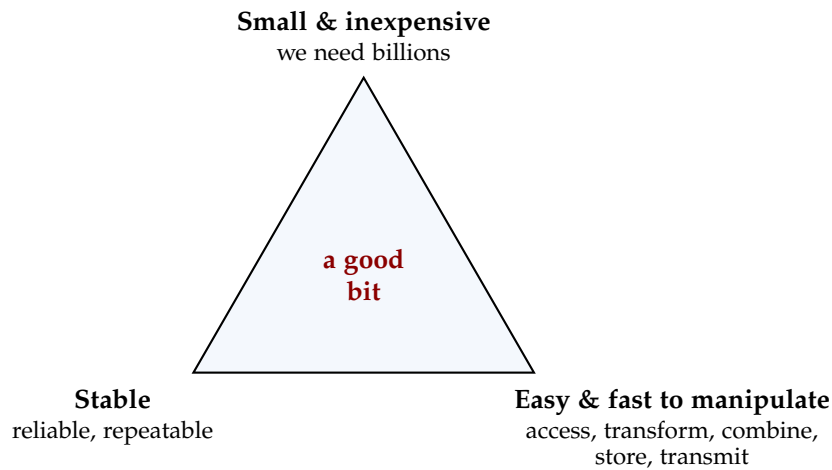
- *small, inexpensive* — as we need lots of them!
- *stable* — reliable and repeatable
- *ease and speed of manipulation* — access, transform, combine, store, transmit

(Illustrated with a stone inscription and a DNA double helix.)

5.1.1 A bit must live in something physical

A **bit** is an abstract choice between two alternatives. To store or process it, some physical quantity must take one of two distinguishable states. The slide's pictures remind us that humans and nature have used many media:

Medium	What encodes the information	Small & cheap?	Stable?	Fast to manipulate?
Stone inscription	carved shapes	no: huge per symbol	extremely (centuries)	no: chisel
DNA	sequence of 4 bases (A, C, G, T) = 2 bits per base	extremely dense	yes (when stored well)	slow to read/write with today's chemistry
Paper	ink marks	moderate	years–centuries	slow
Magnetic disk	direction of magnetisation	yes	years	milliseconds
Electrical voltage	high/low voltage on a wire or capacitor	yes (billions per chip)	yes, if actively re-stored	nanoseconds or less



Intuition : the three requirements pull in different directions

Stone is stable but huge and slow. A tiny charge on a microscopic capacitor is small and fast but leaks away (this is why DRAM must be refreshed thousands of times per second). Every storage and logic technology is a compromise among these three corners, which is another instance of the course's recurring theme: *trade-offs*.

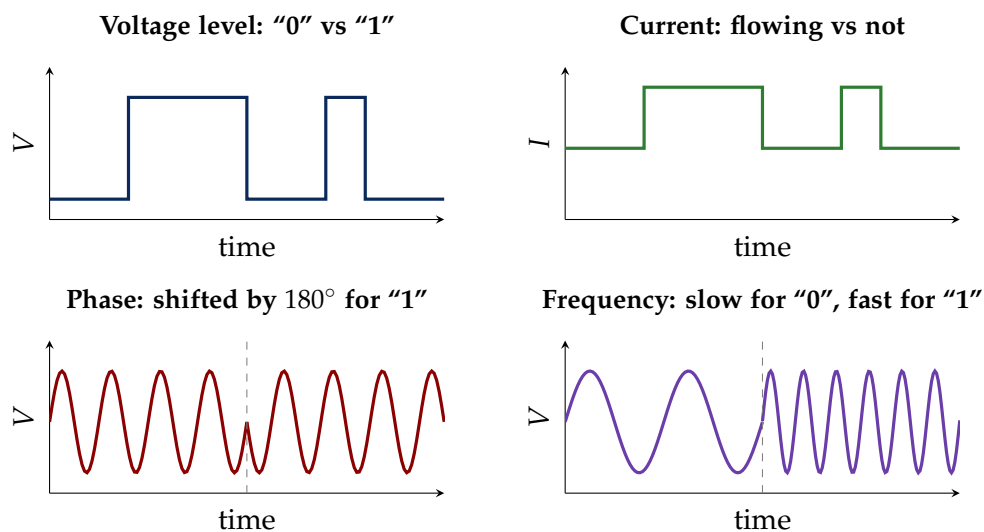
5.2 Using Electrical Phenomena

From the slides:

Using Electrical Phenomena Phenomena associated with charged particles: **Voltage, Current, Phase, Frequency**. We will primarily stick to **voltages** as the primary choice for encoding information.

Voltage pros: easy generation and detection; engineering knowledge; very low power in steady state.

Voltage cons: easily affected by environment; DC connectivity required?; resistive and capacitive effects slow things down.



- **Voltage** (electric potential difference, volts): the level on a wire relative to ground. Used inside essentially every digital chip.

- **Current** (amperes): whether, or how much, charge flows. Used in some interfaces (e.g. current-loop signalling in industrial sensors) and inside certain memory technologies.
- **Phase**: *when* a periodic signal crosses zero relative to a reference. Used in radio (e.g. phase-shift keying in Wi-Fi).
- **Frequency**: *how fast* a signal oscillates. Used in FM radio and old modems (frequency-shift keying).

Phase and frequency encodings shine when signals must travel through the air or long cables; voltage levels shine inside a chip.

5.2.1 Why voltage, and what it costs

Pros	Cons
Easy generation and detection. A transistor's gate responds to voltage directly, and a transistor produces a voltage at its output: the same physical quantity is both input and output, so gates chain naturally.	Easily affected by environment. Nearby switching wires couple noise in (crosstalk), supply voltages droop, temperature shifts transistor behaviour.
Engineering knowledge. Decades of circuit theory and manufacturing know-how exist for voltage-mode circuits.	DC connectivity required? Holding a steady level means an unbroken conductive (DC) path from driver to receiver; you cannot send a steady voltage through a transformer or across a capacitor. (Phase/frequency schemes avoid this.)
Very low power in steady state. Holding a voltage on a MOSFET gate draws almost no current (the gate is insulated). With CMOS, energy is spent mainly when switching.	Resistive and capacitive effects slow things down. Every wire has resistance R and capacitance C ; changing its voltage means moving charge through R into C , which takes time (see below).

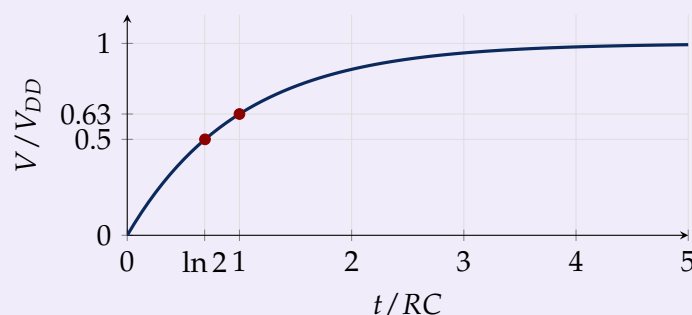
Beyond the Slides : the RC delay behind “resistive and capacitive effects slow things down”

A driver switching a wire from 0 to V_{DD} through resistance R [Ω] into capacitance C [F] obeys

$$V(t) = V_{DD} \left(1 - e^{-t/RC}\right), \quad \tau = RC \quad [s = \Omega \cdot F = (V/A)(C/V) = C/A = s].$$

Derivation. Current into the capacitor is $i = C \frac{dV}{dt}$ and through the resistor is $i = \frac{V_{DD}-V}{R}$. Equate: $RC \frac{dV}{dt} = V_{DD} - V$. Separate variables: $\frac{dV}{V_{DD}-V} = \frac{dt}{RC}$; integrate with $V(0) = 0$: $-\ln \frac{V_{DD}-V}{V_{DD}} = \frac{t}{RC}$, which rearranges to the result.

Time to reach 50% of V_{DD} : $e^{-t/RC} = \frac{1}{2} \Rightarrow t_{50} = RC \ln 2 \approx 0.69 RC$.

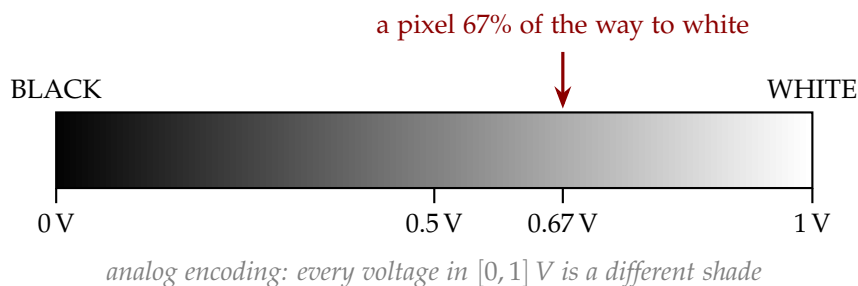


Longer wires have larger R and C , so they are slower. This is one reason chip designers care so much about keeping communicating blocks close together — the “communication unit” of Chapter 2 has a real physical cost.

5.3 Representing Information with Voltage: an Image

From the slides:

Representing Information with Voltage Representation of each $[x, y]$ point in a black-and-white image: 0 V — BLACK; 1 V — WHITE; 0.67 V — 67% gray. *How much information is stored at each point (in bits)?* Our ability to encode information using voltages will clearly be constrained by our ability to reliably and quickly distinguish the voltage at a particular time.



A note on the wording: the slide labels 0.67 V “67% gray”. With 0 V = black and 1 V = white, 0.67 V is a shade whose *brightness* is 67% of white (a light-medium grey). Always check which end a percentage is measured from.

5.3.1 How much information is stored at each point?

The naive answer: infinite. Voltage is a continuous quantity. Between 0 V and 1 V there are infinitely many values, so in principle one point could encode an infinite number of shades — infinitely many bits.

The real answer: limited by how finely we can tell voltages apart. Every real measurement is disturbed by noise, and every real component adds error. If the smallest voltage difference we can *reliably and quickly* distinguish is ΔV , then we can only use levels spaced at least ΔV apart.

Formula : usable levels and information per point

Over a range $V_{\text{range}} = V_{\text{max}} - V_{\text{min}}$ [V] with resolvable step ΔV [V]:

$$N = \left\lfloor \frac{V_{\text{range}}}{\Delta V} \right\rfloor + 1 \quad [\text{levels} = V/V + 1, \text{ dimensionless}], \quad I = \log_2 N \quad [\text{bits per point}].$$

Derivation. Place levels at V_{min} , $V_{\text{min}} + \Delta V$, $V_{\text{min}} + 2\Delta V$, ... up to V_{max} . The number of steps is $V_{\text{range}}/\Delta V$ (rounded down), and there is one more level than steps (fence-post counting). Choosing one of N equally likely levels conveys $\log_2 N$ bits, because n bits can label exactly 2^n things, so labelling N things needs $n = \log_2 N$ (Chapter 7 proves this carefully).

Worked Example

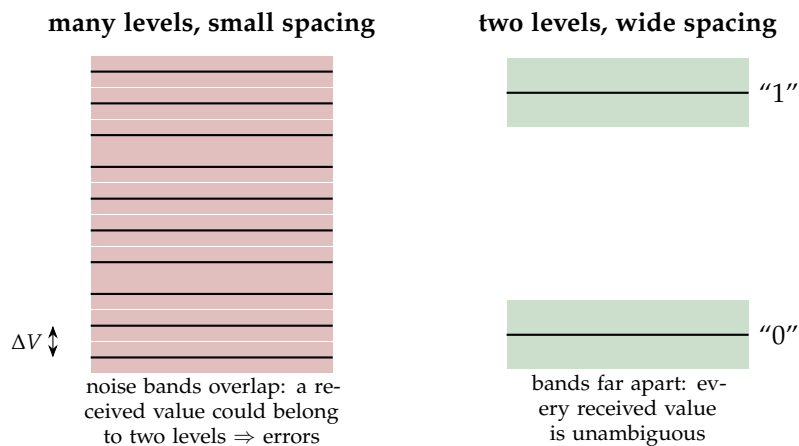
Range 0–1 V.

- $\Delta V = 1 \text{ mV}$: $N = 1/0.001 + 1 = 1001$ levels, $I = \log_2 1001 \approx 9.97$ bits per point.

- $\Delta V = 0.1 \text{ V}$ (a noisy environment): $N = 11$, $I = \log_2 11 \approx 3.46$ bits.
- $\Delta V = 0.5 \text{ V}$: $N = 3 \dots$ and if we insist on generous safety margins, we end up with just **2 levels: 1 bit**. This is where we are heading.

Intuition : a ruler in the rain

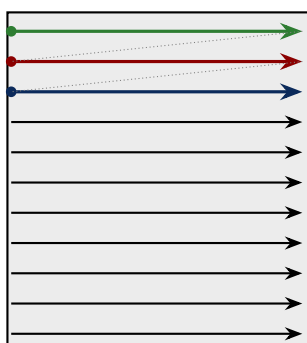
Measuring voltage is like reading a ruler through rain on your glasses. With clear vision you can read millimetres; in a storm you can only tell “short” from “long”. The finer the marks you try to use, the more often you misread. The slide’s closing sentence — information is “constrained by our ability to reliably and quickly distinguish the voltage” — is this idea. And “quickly” matters too: a measurement averaged over a long time can be more precise, but computers need answers in nanoseconds.



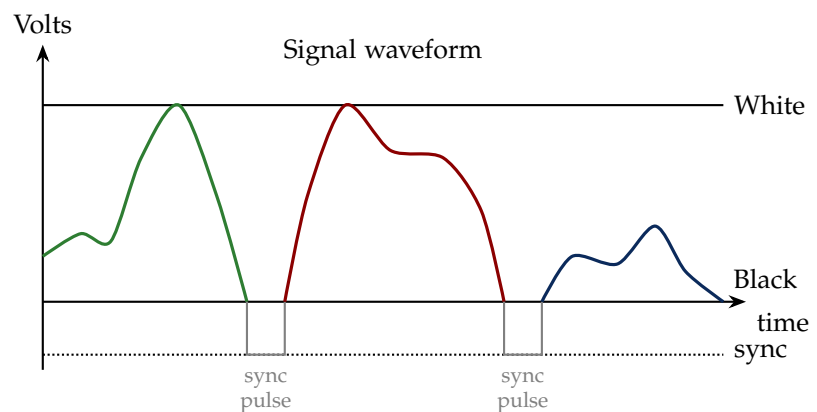
5.4 Using Voltages to Encode a Picture

From the slides:

Using Voltages to Encode a Picture Scan points in some prescribed **raster order** to generate the voltage waveform. The image is crossed by horizontal left-to-right scan lines (first line green, second red, third blue, $\dots$). The signal waveform plots volts vs time between a *Black* and a *White* level; each line’s segment is separated by a short pulse down to a *sync* level, below black.



raster scan of the image



5.4.1 How the encoding works

1. **Serialise the 2-D image into 1-D time.** Read pixels line by line, left to right, top to bottom (the *raster order*, the same order you read a page). Each pixel's brightness becomes a voltage at one instant.
2. **Mark where each line starts.** At the end of each line, drive the signal to a special **sync** level, *below* black. Since no pixel is ever darker than black, the receiver can never confuse a sync pulse with image content. The pulse tells the receiver “start a new line now”.
3. **Receiver reverses the process.** A display sweeps a spot (or updates pixels) in the same raster order, setting each spot's brightness from the voltage, and restarts a line whenever it sees sync.

This is exactly how analog television and old VGA monitors worked.

Formula : raster signalling rate

For an image of $W \times H$ pixels [pixels] shown at F frames per second [frames/s]:

$$R_{\text{pix}} = W \cdot H \cdot F \quad [\text{pixels/s}], \quad t_{\text{pix}} = \frac{1}{R_{\text{pix}}} \quad [\text{s/pixel}], \quad t_{\text{line}} = \frac{1}{H F} \quad [\text{s/line}].$$

(Ignoring the small extra time used by sync and retrace.)

Worked Example

640×480 pixels at 30 frames/s: $R_{\text{pix}} = 640 \times 480 \times 30 = 9\,216\,000$ pixels/s, so each pixel's voltage must be set and read in $t_{\text{pix}} \approx 108.5$ ns. Each line lasts $1/(480 \times 30) \approx 69.4$ μs . The voltage must be generated *and* distinguished within about a tenth of a microsecond — the “quickly” in “reliably and quickly distinguish”.

Key Idea

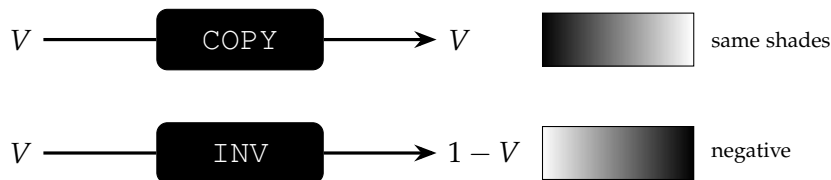
Any information — an image, sound, text — can be turned into a voltage that varies over time. The question the next slides answer is: *what happens when we process such signals with real, imperfect components?*

Computation, Noise, and the Digital Abstraction

6.1 Information Processing = Computation

From the slides:

Information Processing = Computation Two processing blocks act on the image voltage V : COPY outputs V (the same picture); INV outputs $1 - V$ (the photographic negative). “Pre-packaged functionality!” — if such components work individually, the overall system is guaranteed to work reliably.



6.1.1 The functions, precisely

With voltages in $[0, 1]$ V:

$$\text{COPY: } V_{\text{out}} = f_C(V_{\text{in}}) = V_{\text{in}} \quad [V \rightarrow V] \quad (6.1)$$

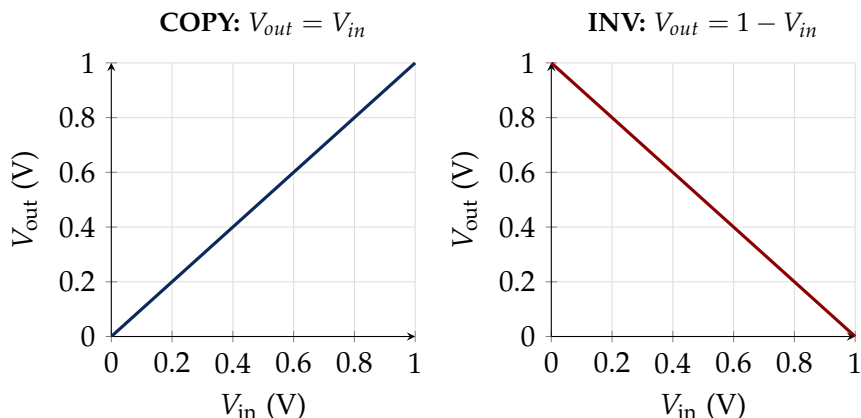
$$\text{INV: } V_{\text{out}} = f_I(V_{\text{in}}) = 1 - V_{\text{in}} \quad [V \rightarrow V; \text{ the "1" is 1 V}] \quad (6.2)$$

Check the INV on the endpoints: black (0 V) $\rightarrow$ 1 V (white); white $\rightarrow$ black; 0.67 V $\rightarrow$ 0.33 V. It maps every shade to its complement, producing the negative image on the slide.

Composing blocks. Connecting blocks in series composes their functions:

$$f_I(f_I(V)) = 1 - (1 - V) = V, \quad f_C(f_I(V)) = 1 - V, \quad f_I(f_C(V)) = 1 - V.$$

So two inverters in a row are, *ideally*, a copy.

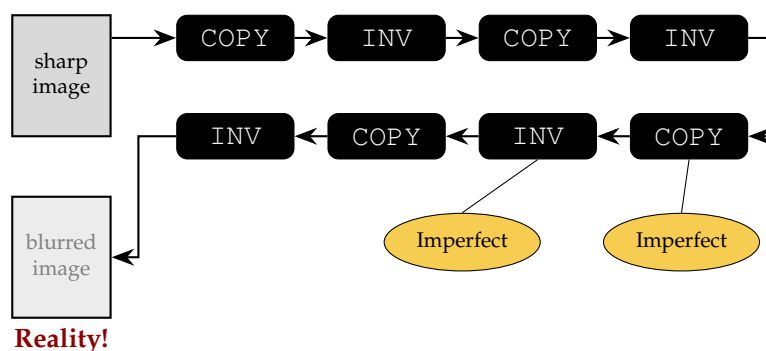


Key Idea : “pre-packaged functionality”

This is abstraction again: design a block once, verify it, then use it as a black box. The slide’s promise — *if the components work individually, the overall system works* — is what makes big systems buildable. The next slide shows that for *analog* blocks this promise is false.

6.2 How About This System? Noise Accumulation**From the slides:**

How About This System? The picture is passed through a long chain: COPY→INV→COPY→INV→COPY→INV→COPY→INV. Ideally the output equals the input (an even number of inversions). The output shown is badly **blurred**: “Reality!” Callouts point at blocks: “Imperfect”.

**6.2.1 Why the picture degrades**

Real components are never exactly the ideal functions. Each one:

- has a slight **gain error** (outputs $0.99V$ or $1.01V$ instead of V),
- has an **offset** (adds a few millivolts),
- has limited **bandwidth** (cannot follow fast changes perfectly, so sharp edges get smoothed — that is the *blur*),
- picks up **noise** from its environment (thermal noise, interference from neighbouring wires).

An analog block has no way to know what the “correct” voltage was, so it faithfully passes along its input’s errors *and adds its own*. Errors **accumulate**.

6.2.2 Quantifying accumulation

Model stage k as ideal function plus an error: $V_k = f(V_{k-1}) + e_k$ [V]. For COPY/INV (slopes ± 1), an error at the input appears with the same magnitude at the output, so after N stages the total error is the sum of the individual errors, $E_N = \sum_{k=1}^N \pm e_k$.

Formula : analog error growth

Worst case (errors all push the same way, each at most ϵ [V]):

$$|E_N| \leq \sum_{k=1}^N |e_k| \leq N\epsilon \quad [\text{V}].$$

Random, independent errors with zero mean and standard deviation σ [V] each:

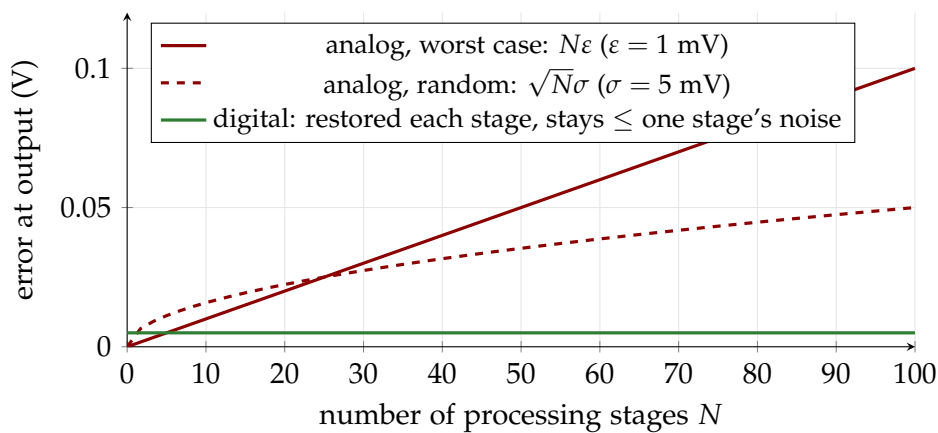
$$\text{Var}(E_N) = \sum_{k=1}^N \text{Var}(e_k) = N\sigma^2 \Rightarrow \sigma_{E_N} = \sqrt{N}\sigma \text{ [V]}.$$

Gain error δ per stage (each stage multiplies the signal by $1 + \delta$):

$$V_N = (1 + \delta)^N V_0 \approx (1 + N\delta) V_0 \text{ for small } N\delta.$$

Derivation of the random case. For independent random variables, variances add: $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$ (the cross-term $2\text{Cov}(X, Y)$ vanishes). Summing N copies of variance σ^2 gives $N\sigma^2$; the standard deviation is its square root. The small- δ gain approximation is the binomial expansion $(1 + \delta)^N = 1 + N\delta + \binom{N}{2}\delta^2 + \dots$ with higher terms dropped.

Every case **grows without bound** as N grows. With $\sigma = 5$ mV per stage, 100 stages give $\sigma_E = 50$ mV and 10 000 stages give 0.5 V — the whole signal range. A processor has millions of gates on some paths through time; with analog signalling it could never work.



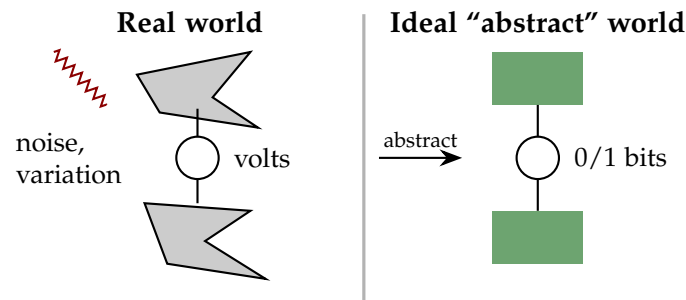
6.3 The Digital Abstraction

From the slides:

The Digital Abstraction **Real world**: two irregular shapes connected by a wire carrying *volts*, disturbed by lightning (“manufacturing variations, noise”). Processing such signals using real-world “non-ideal” components leads to errors, aka noise; such noise keeps accumulating within the system with more analog processing elements.

Ideal “abstract” world: two clean green boxes connected by a wire carrying 0/1 *bits*. Digital processing elements restore noisy input values to legal output values — *signaling errors don’t accumulate in digital systems*. Hence the number of processing elements isn’t limited by noise problems.

Banner: *Neither the real world is digital, nor the components we use to model digital systems are.*



6.3.1 What the abstraction actually says

1. **Reality is analog.** Physical signals are continuous voltages, disturbed by manufacturing variation (no two transistors are identical) and noise.
2. **We choose to interpret them as only two values, 0 and 1,** and we build every component so that it treats any input “close enough” to 0 as exactly 0 and any input “close enough” to 1 as exactly 1.
3. **Each component restores.** Its *output* is always a clean, legal 0 or 1 — even when its input was a bit noisy. Noise is wiped out at every stage instead of being passed on.
4. **Therefore errors do not accumulate,** and we can chain as many components as we like. The system size is no longer limited by noise.

Intuition : Chinese whispers vs a spelling bee

In “Chinese whispers” (telephone), each person repeats what they *heard*, including the distortions, and after twenty people the message is nonsense: analog. Now restrict the message to a single word from a list of two, “YES” or “NO”. Each person hears a slightly mumbled word, decides which of the two it was, and says it *clearly*. Unless someone mumbles so badly that YES sounds like NO, the message survives any number of people: digital. The cost is that you can only send one bit per word.

Key Idea

The digital abstraction **trades information per wire for reliability**. An analog wire could carry “infinitely many” values but degrades; a digital wire carries 1 bit but can be regenerated perfectly an unlimited number of times. When you need more information, you use *more wires* (more bits) rather than finer voltages.

Common Pitfall

The banner is a trap for careless readers: digital does *not* mean the physics is digital. Components are analog devices with voltages that pass continuously through intermediate values during every transition. “Digital” is a *discipline* we impose on how we design and interpret them.

6.4 Using Voltages “Digitally”

From the slides:

Using Voltages “Digitally” **Scheme #1:** a single threshold V_{TH} . $V < V_{TH}$: interpret as 0; $V \geq V_{TH}$: interpret as 1.

Scheme #2: two thresholds V_L and V_H . $V \leq V_L$: interpret as 0; $V \geq V_H$: interpret as 1; between them is the “forbidden” zone.

Encode only one bit of information, 2 states: “0” and “1”. Use the same uniform representation across all components/wires in the digital system.

Scheme #1: one threshold V_{TH}



Scheme #2: two thresholds and a forbidden zone



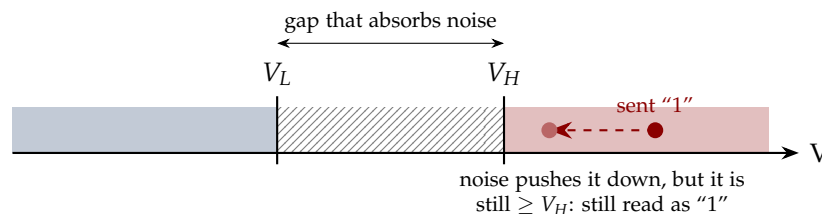
Typo on the slide: the forbidden zone is written “ $V_L > V > V_H$ ”. Since $V_L < V_H$, no voltage satisfies that. It should read $V_L < V < V_H$.

6.4.1 What is wrong with a single threshold?

Scheme #1 looks simpler, but consider a voltage *just* above V_{TH} , say $V_{TH} + 1$ mV. It means “1”. Send it down a wire that adds 2 mV of noise and it may arrive as $V_{TH} - 1$ mV: “0”. Near the threshold, arbitrarily small noise flips the bit. Two components can even disagree about the same voltage, because manufacturing variation means their thresholds are not exactly equal.

6.4.2 How the forbidden zone fixes it

Scheme #2 makes a *rule for senders*: **never produce a voltage in the forbidden zone** (in steady state). Every legal signal is at most V_L or at least V_H . So a signal is always at least $(V_H - V_L)$ away from being confused with the other value. The forbidden zone is a buffer region that absorbs noise.



Key Idea

Use the same uniform representation across all components and wires. If every component agrees on V_L and V_H , any output can drive any input. This uniformity is what lets you snap together gates from a library without re-checking the electrical details every time.

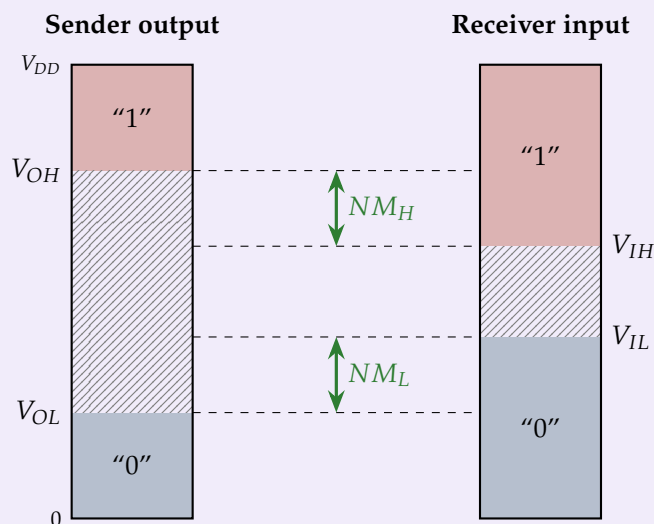
6.4.3 Beyond the slides: noise margins and the static discipline

Beyond the Slides : output thresholds are stricter than input thresholds

Scheme #2 still has a hole: if a sender outputs *exactly* V_H , the tiniest noise pushes it into the forbidden zone. Real logic families therefore use **four** voltages:

Symbol	Rule	Who must obey
V_{OH}	a driver outputting "1" produces $V \geq V_{OH}$	senders
V_{OL}	a driver outputting "0" produces $V \leq V_{OL}$	senders
V_{IH}	a receiver treats any $V \geq V_{IH}$ as "1"	receivers
V_{IL}	a receiver treats any $V \leq V_{IL}$ as "0"	receivers

with $V_{OL} < V_{IL} < V_{IH} < V_{OH}$. Senders are held to a *stricter* standard than receivers: "be conservative in what you send, liberal in what you accept". This rule is called the **static discipline**.



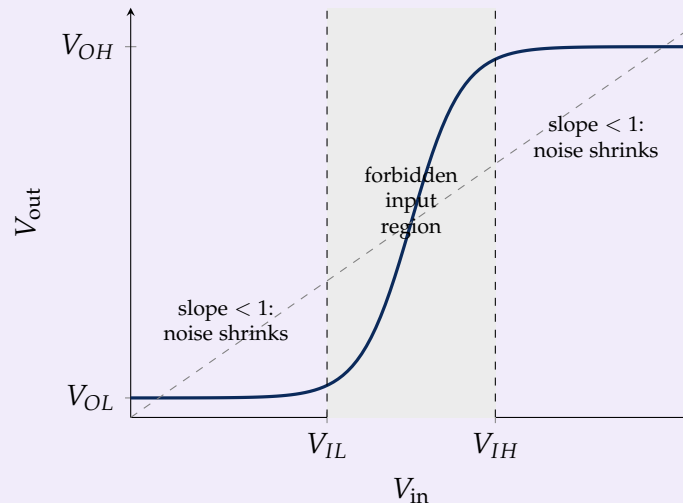
Noise margins are the gaps between what senders guarantee and what receivers require:

$$NM_H = V_{OH} - V_{IH} \quad [\text{V}], \quad NM_L = V_{IL} - V_{OL} \quad [\text{V}].$$

Why this is the tolerable noise. A sender's worst-case "1" is V_{OH} . Noise v_n (negative) arrives at the receiver as $V_{OH} + v_n$. The receiver still reads "1" as long as $V_{OH} + v_n \geq V_{IH}$, i.e. $|v_n| \leq V_{OH} - V_{IH} = NM_H$. Symmetrically for "0": worst case V_{OL} , noise positive, need $V_{OL} + v_n \leq V_{IL}$, so $v_n \leq NM_L$. The system tolerates noise up to $\min(NM_H, NM_L)$ at every single stage, independently of how many stages there are.

Beyond the Slides : how a gate restores — the transfer characteristic

A real buffer (or two inverters in a row) does not have the straight-line COPY characteristic of the idealised slide. Its voltage transfer characteristic (VTC) is an S-curve: flat near both ends and steep in the middle.



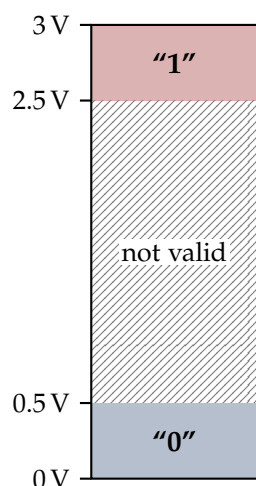
- In the valid input regions ($V_{in} \leq V_{IL}$ or $\geq V_{IH}$) the curve is **flat** (slope magnitude < 1): a noisy input still produces an output very close to the clean level. Input noise is *compressed*. This is the physical mechanism of “restoring noisy inputs to legal outputs”.
- In the forbidden region the curve is **steep** (slope magnitude > 1 , i.e. gain): the gate pushes any in-between input decisively towards one side.
- The dashed line is the ideal analog COPY (slope exactly 1): it neither compresses nor amplifies noise, so noise accumulates.

A gate must have **gain greater than 1** somewhere to be able to restore. That is why digital logic needs active devices (transistors), not just passive resistors.

6.5 Binary Digital System

From the slides:

Binary Digital System **Digital system**: finite number of values. **Binary (base 2) system**: uses two states or values. Basic unit of information: the **binary digit, or bit** — two values, 0 and 1, represented by voltages. Key advantage: **efficient circuits** (cheap, small, fast, low power). Diagram: a voltage axis with 3 V and 2.5 V marking the “1” band, 0.5 V and 0 V marking the “0” band.



One reading of the figure (static discipline):

drivers output $\approx 3\text{ V}$ for “1” and $\approx 0\text{ V}$ for “0”; receivers accept $\geq 2.5\text{ V}$ as “1” and $\leq 0.5\text{ V}$ as “0”.

Then $NM_H = 3 - 2.5 = 0.5\text{ V}$ and $NM_L = 0.5 - 0 = 0.5\text{ V}$:

up to 0.5 V of noise is tolerated per stage.

6.5.1 Why binary rather than 3, 10 or 100 levels?

- **Largest possible spacing.** With a fixed voltage range, two levels leave the widest forbidden zone, so the best noise immunity.
- **Simplest circuits.** A transistor is naturally either ON or OFF. A two-state device needs only to decide “above or below”, which is cheap, small, fast and low-power — the “efficient circuits” of the slide.
- **Matches logic.** Two values map directly onto TRUE/FALSE, which lets Boolean algebra describe circuits exactly (Chapter 7).

Formula : why more levels cost noise immunity

With range V_{range} divided into L levels, adjacent levels are $V_{\text{range}}/(L - 1)$ apart. A received value is decoded to the nearest level, so it is decoded correctly as long as noise stays below half the spacing:

$$v_{n,\text{max}} = \frac{V_{\text{range}}}{2(L - 1)} \quad [\text{V}].$$

For $V_{\text{range}} = 3 \text{ V}$: $L = 2$ tolerates 1.5 V (ideal, before any forbidden-zone allowance); $L = 4$ tolerates 0.5 V; $L = 10$ only 0.167 V.

Beyond the Slides

Multi-level signalling *is* used where wires are expensive and speed is king: modern flash memory stores 3–4 bits per cell by distinguishing 8–16 charge levels, and some high-speed links use 4-level signalling (PAM-4). They pay for it with error-correcting codes and careful analog design. Inside logic, binary wins.

Bits, Number Representation, and Boolean Logic

7.1 Can We Represent More Than 2 Values?

From the slides:

Can We Represent More Than 2 Values? Yes — just use multiple bits. A collection of 2 bits gives 4 possible values: 00, 01, 10, 11. A collection of 3 bits gives 8 possible values: 000, 001, 010, 011, 100, 101, 110, 111. A collection of n bits gives 2^n possible values.

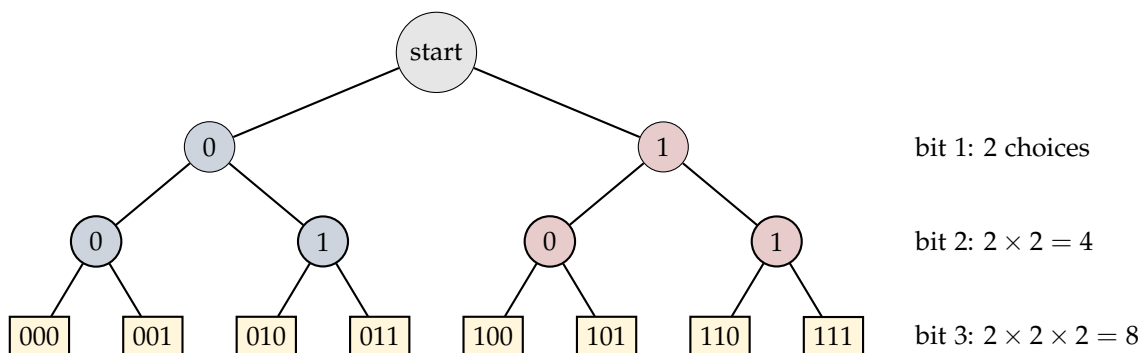
7.1.1 Why n bits give 2^n values

Formula : counting bit patterns

$$N(n) = 2^n \quad [\text{distinct patterns; } n \text{ in bits}]$$

Proof 1 (multiplication principle). Each bit independently takes one of 2 values. The number of ways to choose n independent things, each with 2 options, is $2 \times 2 \times \cdots \times 2$ (n times) $= 2^n$.

Proof 2 (induction). One bit: 2 patterns (0, 1), so $N(1) = 2 = 2^1$. Suppose n bits give 2^n patterns. An $(n + 1)$ -bit pattern is a 0 followed by any n -bit pattern, or a 1 followed by any n -bit pattern: $N(n + 1) = 2^n + 2^n = 2^{n+1}$. By induction, $N(n) = 2^n$ for all $n \geq 1$.



Each level of the tree doubles the number of leaves: every extra bit *doubles* the number of values you can represent.

7.1.2 The inverse question: how many bits do I need?

To give N different things distinct labels we need $2^n \geq N$, i.e. $n \geq \log_2 N$. Since n is a whole number:

Formula : bits required

$$n_{\min} = \lceil \log_2 N \rceil \quad [\text{bits}]$$

n	2^n	example	N items	$\lceil \log_2 N \rceil$	example
1	2	on/off	7	3	days of week
4	16	a hex digit	26	5	English letters
8	256	a byte, one grey level	60	6	minutes in an hour
10	1 024	$\approx$ "1K"	100	7	exam marks 0–99
16	65 536		1 000	10	
32	$\approx 4.29 \times 10^9$	IPv4 addresses	10^6	20	

Intuition : connecting back to the image

The analog image encoded each pixel's shade as one voltage with many possible values (and suffered from noise). The digital way: use *several wires*, each carrying one robust bit. Eight bits per pixel give $2^8 = 256$ grey levels — plenty for the eye — and every one of those bits can be regenerated perfectly. We replaced "many fragile levels on one wire" with "two robust levels on many wires".

Common Pitfall

Don't confuse *number of values* with *largest value*. n unsigned bits represent 2^n values, from 0 to $2^n - 1$. So 8 bits reach 255, not 256.

7.2 Positional Number Representation

From the slides:

Positional Number Representation Recall positional notation for decimal numbers: $329 = 3 \times 100 + 2 \times 10 + 9 \times 1$ (weights $10^2, 10^1, 10^0$; base 10). Similar positional system for binary: $101_2 = 1 \times 4 + 0 \times 2 + 1 \times 1 = 5$ (weights $2^2, 2^1, 2^0$; base 2). We will study computer arithmetic in much more detail later.

	10^2	10^1	10^0	
Base 10	3	2	9	$= 3 \cdot 100 + 2 \cdot 10 + 9 \cdot 1 = 329$
	2^2	2^1	2^0	
Base 2	1	0	1	$= 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 5$

weight $\times$ base for each step left

Formula : value of a positional numeral

An n -digit numeral $d_{n-1}d_{n-2} \dots d_1d_0$ in base b , with each digit $0 \leq d_i \leq b - 1$, has value

$$V = \sum_{i=0}^{n-1} d_i b^i \quad [\text{dimensionless number}].$$

Digit i (counting from 0 at the right) has weight b^i .

Why this works (and why it is unique). The largest value that the lower i digits can reach is

$$\sum_{j=0}^{i-1} (b-1)b^j = (b-1) \cdot \frac{b^i - 1}{b-1} = b^i - 1,$$

using the geometric series $\sum_{j=0}^{i-1} b^j = \frac{b^i - 1}{b-1}$. That is exactly one less than the weight of digit i . So the lower digits can never “overflow” into the next position: every number from 0 to $b^n - 1$ has exactly one representation. For $b = 2$ this gives the pitfall above: the maximum n -bit value is $2^n - 1$.

Worked Example : binary to decimal

$$110101_2 = 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 32 + 16 + 0 + 4 + 0 + 1 = 53.$$

Beyond the Slides : decimal to binary by repeated division

Divide by 2 repeatedly; the remainders, read from last to first, are the bits. Example: 53.

n	$\div 2$	quotient	remainder
53		26	1 $\leftarrow d_0$ (LSB)
26		13	0
13		6	1
6		3	0
3		1	1
1		0	1 $\leftarrow d_5$ (MSB)

Reading upward: $53 = 110101_2$ (matches the example above). Why it works: $V = d_0 + 2(d_1 + 2(d_2 + \dots))$, so $V \bmod 2 = d_0$ and $\lfloor V/2 \rfloor$ is the number formed by the remaining bits. You will do much more of this, plus negative numbers and fractions, in the computer-arithmetic part of Module 1.

7.3 Digital Logic → Digital Circuits

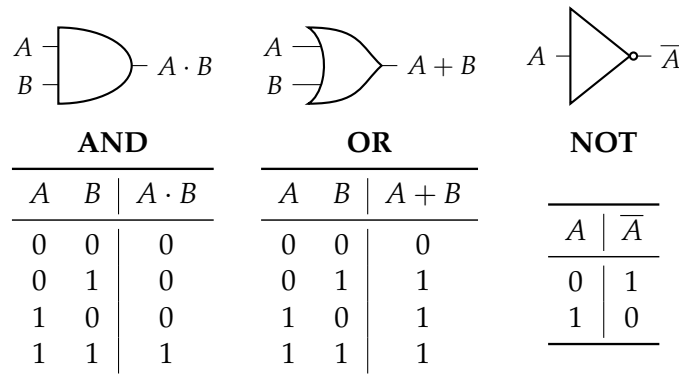
From the slides:

Digital Logic → Digital Circuits **Boolean algebra** (portrait of George Boole) with truth-table gate pictures for AND, OR, NOT. 1854: George Boole shows that *logic is math, not just philosophy!* Boolean algebra: the mathematics of **binary** values.

Key link between logic and circuits (portrait of Claude Shannon): logic gates + the vacuum tube (Lee de Forest, 1906) = *Digital Electronics*. Despite the existence of relays and vacuum tubes in 1906, digital electronics did not emerge for thirty more years! Claude Shannon notices similarities between Boolean algebra and electronic telephone switches. Shannon’s 1937 MIT Master’s thesis introduces the world to **binary digital electronics**.

7.3.1 Boolean algebra: logic as arithmetic on $\{0, 1\}$

George Boole (1854, *The Laws of Thought*) showed that reasoning with TRUE/FALSE statements obeys algebraic rules, just like arithmetic with numbers. Represent FALSE by 0 and TRUE by 1. The three basic operations:



- **AND** ($A \cdot B$): 1 only if *both* inputs are 1. Arithmetically it is ordinary multiplication restricted to $\{0, 1\}$.
- **OR** ($A + B$): 1 if *at least one* input is 1. Like addition, except $1 + 1 = 1$ (there is no “2” in Boolean algebra).
- **NOT** ($\bar{A}$): flips the value, $\bar{A} = 1 - A$ — exactly the digital version of the slide’s INV block!

Intuition : INV was NOT all along

The analog INV computed $1 - V$ for any $V \in [0, 1]$. Restrict V to $\{0, 1\}$ and it becomes Boolean NOT. The digital abstraction and Boolean algebra fit together perfectly: once signals are only 0 or 1, circuits compute Boolean functions.

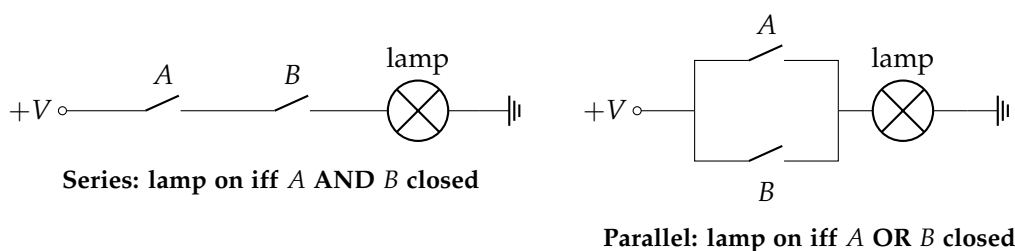
Beyond the Slides : a few Boolean identities you will use in Module 1

Law	AND form	OR form
Identity	$A \cdot 1 = A$	$A + 0 = A$
Null	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotent	$A \cdot A = A$	$A + A = A$
Complement	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$
Involution	$\bar{\bar{A}} = A$ (two inverters = copy)	
De Morgan	$\overline{A \cdot B} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A} \cdot \bar{B}$

Each can be proved by checking all input combinations in a truth table (*perfect induction*), since there are only finitely many.

7.3.2 Shannon: switches compute Boolean functions

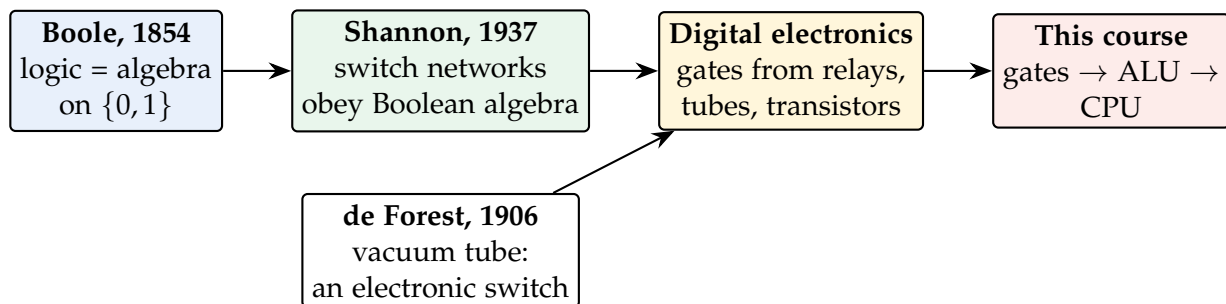
Relays (electrically controlled switches, used in telephone exchanges) and vacuum tubes (Lee de Forest’s 1906 triode, which let a small voltage control a larger current) existed early in the 20th century. Yet for about thirty years nobody had a *systematic* way to design switching circuits. Claude Shannon, in his 1937 MIT master’s thesis (*A Symbolic Analysis of Relay and Switching Circuits*), noticed that networks of switches obey Boolean algebra:



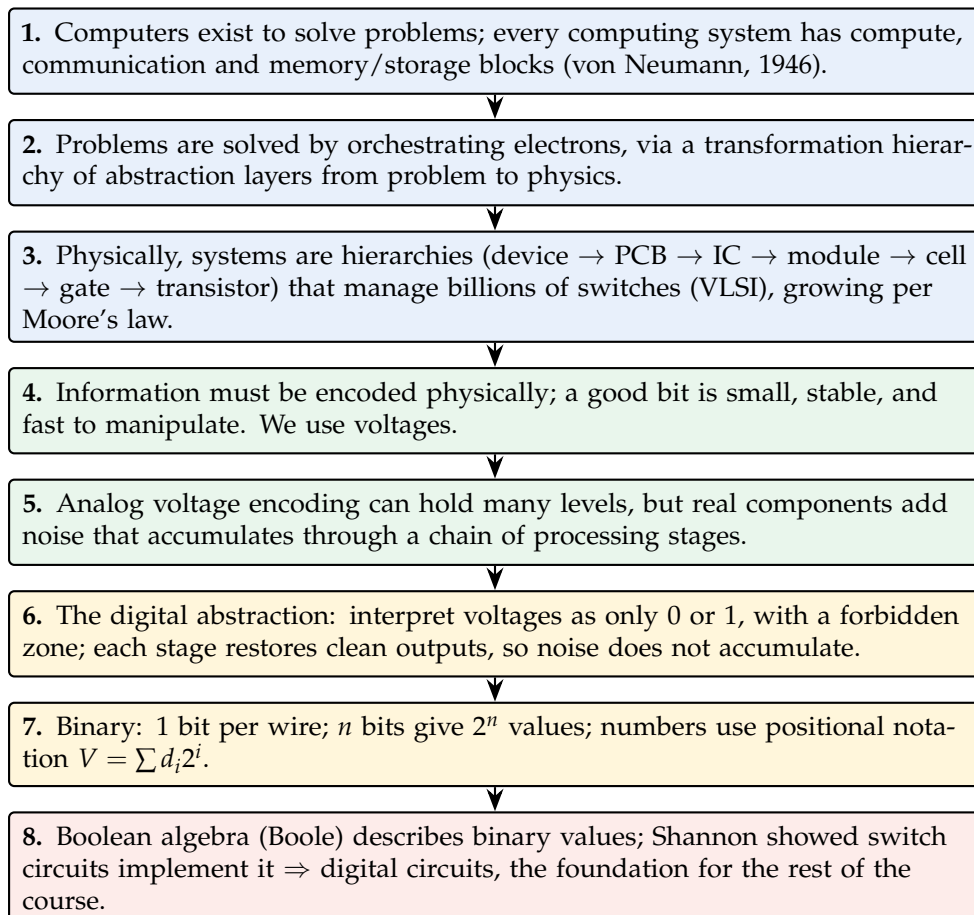
Switch network	Boolean operation	Why
Two switches in series	AND	current flows only if every switch in the path is closed
Two switches in parallel	OR	current flows if any one path is closed
Normally-closed switch	NOT	conducts when the control is 0

Key Idea

Shannon's insight turned circuit design from trial and error into *algebra*. Write the desired behaviour as a Boolean expression, simplify it with algebraic laws, and then build it directly as switches. Replace relays with vacuum tubes, then with transistors (Chapter 3), and you have modern digital electronics. Everything in Modules 2–4 — adders, ALUs, registers, processors — is designed this way.



7.4 Module 1 Summary: the Whole Week in One Chain of Reasoning



Part III

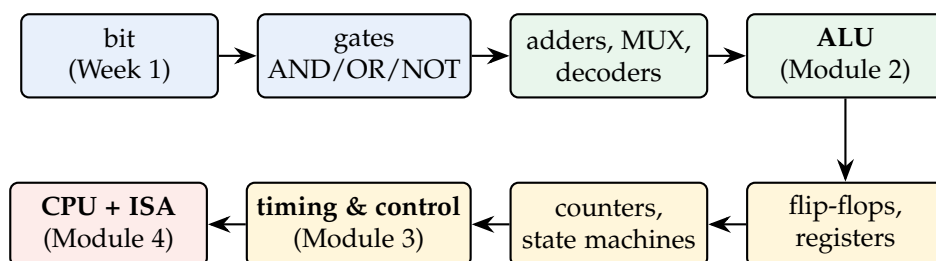
Looking Ahead

Chapter 8

Preview: Where Week 1 Leads (Modules 2–4)

This chapter is a short map, not examinable Week 1 content. It shows how each Week 1 idea gets used, so the foundations feel purposeful.

8.1 The path from a bit to a CPU



Module 1 (rest). Boolean algebra in depth: laws, canonical forms (sum of products), simplification (Karnaugh maps), universal gates (NAND, NOR). Computer arithmetic: binary, hexadecimal, signed numbers (two's complement), binary addition and overflow.

Module 2: combinational logic → ALU. Circuits whose output depends only on current inputs: half/full adders, ripple-carry adders, multiplexers, decoders, comparators. Combine them into an ALU that performs a selected arithmetic or logic operation.

Module 3: sequential logic → timing and control. Circuits with *memory*: latches, flip-flops, registers, counters, and finite state machines driven by a clock. These sequence the CPU's operations.

Module 4: processor design and ISA. The instruction set (the HW/SW contract from Chapter 3), datapath and control, the fetch–decode–execute cycle of the von Neumann model (Chapter 2).

8.2 Week 1 idea → later use

Week 1 idea	Used later for
Compute, communication and memory blocks	datapath (ALU), buses, register file and memory in the CPU of Module 4
Transformation hierarchy	understanding where the ISA sits and why compilers and hardware co-design matters
Physical hierarchy, cells	gate-level design using libraries of standard building blocks
Moore's law	why we can afford parallel adders, caches, multiple cores
Noise, forbidden zone, restoration	why every gate output is a clean 0/1 you can safely feed into any number of later gates
RC delay (beyond box)	propagation delay of gates, critical paths, maximum clock speed in Module 3
2^n values	sizes of decoders, multiplexer select lines, address spaces, register widths
Positional notation	binary arithmetic, adders, two's complement
Boolean algebra + Shannon	designing and simplifying every circuit in Modules 2–4

Part IV

Practice

Chapter 9

Numerical Problems with Worked Solutions

Try each problem before reading the solution. Units are tracked in grey brackets.

9.1 Bits and counting

Problem 1 *How many bits?*

How many bits are needed to give a unique binary code to (a) 50 students, (b) 1 000 products, (c) exactly 4 096 memory locations, (d) the 26 capital letters plus 10 digits?

Solution

Use $n_{\min} = \lceil \log_2 N \rceil$.

(a) $2^5 = 32 < 50 \leq 64 = 2^6 \Rightarrow n = 6$ bits.

(b) $2^9 = 512 < 1000 \leq 1024 = 2^{10} \Rightarrow n = 10$ bits.

(c) $4096 = 2^{12}$ exactly $\Rightarrow n = 12$ bits (no codes wasted).

(d) $N = 36$; $2^5 = 32 < 36 \leq 64 \Rightarrow n = 6$ bits (28 codes unused).

Problem 2 *Range of an n -bit number*

A sensor outputs a 12-bit unsigned number. How many distinct values can it report, and what are the smallest and largest?

Solution

Distinct values: $2^{12} = 4096$. Smallest: 0 (all zeros). Largest: $2^{12} - 1 = 4095$ (all ones), from the geometric-series result $\sum_{i=0}^{11} 2^i = 2^{12} - 1$.

Problem 3 *Largest value by the geometric series*

Show that $11111_2 = 2^5 - 1$ both by direct addition and using the formula for the maximum n -bit value.

Solution

Direct: $16 + 8 + 4 + 2 + 1 = 31$. Formula: $\sum_{i=0}^4 1 \cdot 2^i = \frac{2^5-1}{2-1} = 31 = 2^5 - 1$. ✓

Problem 4 *Memory addresses*

(a) A memory has 65 536 locations. How many address bits are needed? (b) A byte-addressable memory holds 1 GiB (2^{30} bytes). How many address bits?

Solution

(a) $65\,536 = 2^{16} \Rightarrow 16$ bits. (b) 2^{30} bytes $\Rightarrow 30$ bits. Each extra address bit doubles the addressable memory (Chapter 7).

Problem 5 *Truth tables and Boolean functions*

(a) How many rows does the truth table of a 3-input function have? (b) How many *different* Boolean functions of 2 inputs exist?

Solution

(a) Each row is one input pattern: $2^3 = 8$ rows.
 (b) A function is defined by choosing its output (0 or 1) in each of the $2^2 = 4$ rows. That is 4 independent binary choices: $2^4 = 16$ functions. In general there are $2^{(2^n)}$ Boolean functions of n inputs (AND, OR, XOR, NAND, ... are 6 of the 16 two-input ones).

9.2 Positional number representation**Problem 6** *Binary to decimal*

Convert (a) 101101_2 , (b) 11111111_2 , (c) 1000000_2 to decimal.

Solution

- (a) $1 \cdot 32 + 0 \cdot 16 + 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 45$.
 (b) $128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255 = 2^8 - 1$.
 (c) Only bit 6 is set: $2^6 = 64$.

Problem 7 *Decimal to binary*

Convert (a) 100, (b) 37 to binary by repeated division.

Solution

(a) $100 \rightarrow 50$ r0; $50 \rightarrow 25$ r0; $25 \rightarrow 12$ r1; $12 \rightarrow 6$ r0; $6 \rightarrow 3$ r0; $3 \rightarrow 1$ r1; $1 \rightarrow 0$ r1. Reading remainders bottom-up: 1100100_2 . Check: $64 + 32 + 4 = 100$. ✓
 (b) $37 \rightarrow 18$ r1; $18 \rightarrow 9$ r0; $9 \rightarrow 4$ r1; $4 \rightarrow 2$ r0; $2 \rightarrow 1$ r0; $1 \rightarrow 0$ r1. Bottom-up: 100101_2 . Check: $32 + 4 + 1 = 37$. ✓

Problem 8 *Positional notation in another base*

Evaluate 312_4 in decimal, then convert the result back to base 4.

Solution

$312_4 = 3 \cdot 4^2 + 1 \cdot 4^1 + 2 \cdot 4^0 = 48 + 4 + 2 = 54$.
 Back: $54 \div 4 = 13$ r2; $13 \div 4 = 3$ r1; $3 \div 4 = 0$ r3. Bottom-up: 312_4 . ✓ The same positional rule $V = \sum d_i b^i$ works for every base.

9.3 Representing information

Problem 9 *Grey-scale image storage*

A 640×480 grey-scale image uses 8 bits per pixel. How many grey levels are possible, and how much storage does one image need in bits, bytes and KiB?

Solution

Grey levels: $2^8 = 256$.

Pixels: $640 \times 480 = 307\,200$ [pixels]. Bits: $307\,200 \times 8 = 2\,457\,600$ [bits = pixels $\times$ bits/pixel].

Bytes: $2\,457\,600/8 = 307\,200$ [bytes]. KiB: $307\,200/1024 = 300$ KiB.

Problem 10 *Colour video data rate*

A 1920×1080 colour video uses 24 bits per pixel at 30 frames/s. Find the size of one frame and the raw data rate.

Solution

Frame: $1920 \times 1080 \times 24 = 49\,766\,400$ bits = $6\,220\,800$ bytes ≈ 5.93 MiB (dividing by $2^{20} = 1\,048\,576$).

Rate: $49\,766\,400 \times 30 = 1\,492\,992\,000$ bits/s ≈ 1.49 Gbit/s [bits/frame $\times$ frames/s]. This is why video is compressed.

Problem 11 *Information in an analog voltage*

A voltage ranges over 0–1 V and the smallest reliably distinguishable step is 10 mV. How many levels, and how many bits per sample?

Solution

$N = \lfloor V_{\text{range}}/\Delta V \rfloor + 1 = \lfloor 1/0.01 \rfloor + 1 = 101$ levels [V/V, dimensionless].

$I = \log_2 101 \approx 6.66$ bits. (A 6-bit code would use only 64 of the levels; 7 bits would be needed to label all 101.)

Problem 12 *Required resolution*

To store 8 bits in a single analog voltage between 0 and 1 V, how finely must you distinguish voltages?

Solution

8 bits $\Rightarrow N = 256$ levels $\Rightarrow 255$ gaps across 1 V: $\Delta V = 1/255 \approx 3.92$ mV. Every stage's noise must stay well under half of this (≈ 2 mV) — very hard in a noisy chip, which is why we don't do it.

Problem 13 *DNA as storage*

DNA uses 4 bases. How many bits are stored in a strand of 1 000 bases?

Solution

Each base picks 1 of 4 values: $\log_2 4 = 2$ bits/base. $1000 \times 2 = 2000$ bits = 250 bytes.

Problem 14 *Raster signalling*

A 1024×768 display refreshes at 60 frames/s. Ignoring sync time, how long does each pixel and each line last?

Solution

$$R_{\text{pix}} = 1024 \times 768 \times 60 = 47\,185\,920 \text{ pixels/s. } t_{\text{pix}} = 1/R_{\text{pix}} \approx 21.2 \text{ ns [s/pixel]. } t_{\text{line}} = 1/(768 \times 60) \approx 21.7 \mu\text{s [s/line].}$$

9.4 Noise and the digital abstraction**Problem 15** *Random noise accumulation*

Each analog stage adds independent noise with $\sigma = 2$ mV. (a) After how many stages does the total noise reach $\sigma_{\text{total}} = 100$ mV? (b) With worst-case error $\varepsilon = 2$ mV per stage all in the same direction, how many stages reach 100 mV?

Solution

- (a) $\sigma_{\text{total}} = \sqrt{N}\sigma \Rightarrow \sqrt{N} = 100/2 = 50 \Rightarrow N = 2500$ stages.
 (b) $N\varepsilon = 100 \Rightarrow N = 50$ stages. Correlated errors are much worse than random ones.

Problem 16 *Gain error*

Each stage has a gain error of +1%. By how much is the signal off after 10 stages? After 100? Compare with the linear approximation.

Solution

Exact: $(1.01)^{10} \approx 1.1046 \Rightarrow +10.5\%$. Approximation $1 + N\delta = 1.10 \Rightarrow +10\%$: good.
 Exact: $(1.01)^{100} \approx 2.705 \Rightarrow +170\%$. Approximation: $1 + 100(0.01) = 2$, i.e. +100%: poor, because $N\delta = 1$ is not small. Either way, the signal is ruined.

Problem 17 *How analog degenerates to binary*

Signals span 0–1 V; each stage adds random noise $\sigma = 1$ mV. Suppose levels must be at least $6\sigma_{\text{total}}$ apart to be told apart reliably. How many bits per wire remain usable after 100 stages? After 10 000?

Solution

After 100 stages: $\sigma_{\text{total}} = \sqrt{100} \times 1 = 10$ mV; spacing 60 mV; $N = \lfloor 1/0.06 \rfloor + 1 = 17$ levels $\Rightarrow \log_2 17 \approx 4.09$ bits.
 After 10 000 stages: $\sigma_{\text{total}} = 100$ mV; spacing 0.6 V; $N = \lfloor 1/0.6 \rfloor + 1 = 2$ levels $\Rightarrow 1$ bit.
 The longer the chain, the fewer levels survive, until only two remain. The digital abstraction simply chooses two levels from the start and restores them at every stage.

Problem 18 *Noise margins of a logic family*

A logic family has $V_{OH} = 2.7$ V, $V_{OL} = 0.4$ V, $V_{IH} = 2.0$ V, $V_{IL} = 0.8$ V. Find the noise margins and the maximum noise tolerated.

Solution

$NM_H = V_{OH} - V_{IH} = 2.7 - 2.0 = 0.7$ V. $NM_L = V_{IL} - V_{OL} = 0.8 - 0.4 = 0.4$ V. Maximum tolerable noise (either value): $\min(0.7, 0.4) = 0.4$ V.

Problem 19 *The slide's 3 V system*

Using the binary-system slide (drivers at 3 V and 0 V, receivers accept ≥ 2.5 V as 1 and ≤ 0.5 V as 0): (a) find NM_H , NM_L . (b) A wire picks up 0.6 V of noise. Is a transmitted "1" still safe?

Solution

(a) $NM_H = 3 - 2.5 = 0.5$ V; $NM_L = 0.5 - 0 = 0.5$ V.

(b) Worst case, noise pulls the "1" down: $3 - 0.6 = 2.4$ V < 2.5 V. It lands in the forbidden zone, so it is *not* guaranteed to be read as 1. Noise exceeds the margin.

Problem 20 *Can family A drive family B?*

Family A: $V_{OH} = 2.4$ V, $V_{OL} = 0.4$ V. Family B: $V_{IH} = 3.5$ V, $V_{IL} = 1.5$ V. Can an A output drive a B input reliably?

Solution

$NM_H = V_{OH}^A - V_{IH}^B = 2.4 - 3.5 = -1.1$ V < 0 . **No:** A's worst-case "1" is already inside B's forbidden zone, even with zero noise. $NM_L = V_{IL}^B - V_{OL}^A = 1.5 - 0.4 = 1.1$ V (fine). One failing margin is enough to make the interface unreliable. This is why the slide insists on a *uniform representation across all components*.

Problem 21 *Levels vs noise tolerance*

A 1.8 V supply range is divided into L equally spaced levels, decoded to the nearest level. Find the maximum tolerable noise for $L = 2, 4, 8$.

Solution

$$v_{n,\max} = \frac{V_{\text{range}}}{2(L-1)}. \quad L = 2: 1.8/2 = 0.9 \text{ V. } L = 4: 1.8/6 = 0.3 \text{ V. } L = 8: 1.8/14 \approx 0.129 \text{ V.}$$

Tripling the information from 1 to 3 bits per wire cuts noise tolerance by a factor of 7.

Problem 22 *RC delay*

A gate drives a wire with $R = 1$ k Ω into $C = 100$ fF. Find τ , the 50% delay, and the time to reach 90%.

Solution

$$\tau = RC = 10^3 \Omega \times 100 \times 10^{-15} \text{ F} = 10^{-10} \text{ s} = 100 \text{ ps} \quad [\Omega \cdot \text{F} = \text{s}].$$

$$50\%: t = \tau \ln 2 \approx 69.3 \text{ ps. } 90\%: 1 - e^{-t/\tau} = 0.9 \Rightarrow t = \tau \ln 10 \approx 230 \text{ ps.}$$

9.5 Hierarchy and Moore's law

Problem 23 *Transistors through a hierarchy*

A design uses 4 transistors per gate, 8 gates per cell, 5 000 cells per module and 10 modules per IC. A product has 12 such ICs on each of 3 PCBs. How many transistors in one IC and in the product?

Solution

Per IC: $4 \times 8 \times 5000 \times 10 = 1.6 \times 10^6$. Product: $1.6 \times 10^6 \times 12 \times 3 = 5.76 \times 10^7$ transistors.

Problem 24 *Slide's hierarchy bounds*

Using the slide's ranges (2–8 transistors/gate, 2–16 gates/cell, 1K–10K cells/module, 8–10 modules/IC), find the minimum and maximum transistors per IC.

Solution

Min: $2 \times 2 \times 1000 \times 8 = 32\,000 = 3.2 \times 10^4$. Max: $8 \times 16 \times 10\,000 \times 10 = 1.28 \times 10^7$.

Problem 25 *Predicting with Moore's law*

Starting from the 4004 (1971, 2 300 transistors) and $T_d = 2$ years, predict the transistor count in 1991.

Solution

$(1991 - 1971)/2 = 10$ doublings: $N = 2300 \times 2^{10} = 2300 \times 1024 = 2\,355\,200 \approx 2.4 \times 10^6$. (Intel's i486, 1989, had about 1.2×10^6 : the right order of magnitude.)

Problem 26 *Doubling time from data*

The 8086 (1978) had about 29 000 transistors; the Pentium 4 (2000) about 42 000 000. Estimate T_d .

Solution

$N_2/N_1 = 42 \times 10^6/29\,000 \approx 1448$. $\log_2 1448 = \ln 1448 / \ln 2 \approx 7.278/0.693 \approx 10.5$ doublings.
 $T_d = (2000 - 1978)/10.5 = 22/10.5 \approx 2.1$ years.

Problem 27 *Time for a 1000× increase*

With $T_d = 2$ years, how long does a 1000× increase take?

Solution

$2^{\Delta t/T_d} = 1000 \Rightarrow \Delta t = T_d \log_2 1000 = 2 \times 9.966 \approx 19.9$ years (≈ 20 years).

Problem 28 *Annual growth for a different doubling time*

If density doubled every 18 months, what would the annual growth factor be?

Solution

$$2^{1/T_d} = 2^{1/1.5} = 2^{0.667} \approx 1.587 \Rightarrow \approx 58.7\% \text{ per year (vs 41.4\% for } T_d = 2).$$

Problem 29 *Feature-size shrink*

A process shrinks from 90 nm to 45 nm features. By what factor does transistor density increase? How many “0.7×” generations is that?

Solution

$D_2/D_1 = (L_1/L_2)^2 = (90/45)^2 = 4$. Each 0.7× generation doubles density, so $4 = 2^2 \Rightarrow 2$ generations (90 → 63 → 44 nm).

9.6 Why we compute

Problem 30 *Projectile range, time and height*

A projectile leaves at $v_0 = 100$ m/s, $\theta = 30^\circ$, $g = 9.8$ m/s², no drag. Find range, time of flight and maximum height.

Solution

$$R = v_0^2 \sin 2\theta / g = 10^4 \times \sin 60^\circ / 9.8 = 8660 / 9.8 \approx 884 \text{ m} \quad [\text{m}].$$

$$t_{\text{flight}} = 2v_0 \sin \theta / g = 2 \times 100 \times 0.5 / 9.8 \approx 10.2 \text{ s} \quad [\text{s}].$$

Max height: at the top $v_y = 0$: $v_0 \sin \theta - gt = 0 \Rightarrow t_{\text{top}} = v_0 \sin \theta / g$. Then $H = v_0 \sin \theta t_{\text{top}} -$

$$\frac{1}{2}gt_{\text{top}}^2 = \frac{v_0^2 \sin^2 \theta}{2g} = \frac{10^4 \times 0.25}{19.6} \approx 127.6 \text{ m}.$$

Problem 31 *One numerical time step*

Without drag, a ball starts at the origin with velocity (10, 10) m/s. Using the update rule of Chapter 2 (update velocities, then positions) with $\Delta t = 0.1$ s and $g = 9.8$ m/s², find the position after one step and compare with the exact answer.

Solution

$$v_x = 10, v_y = 10 - 9.8 \times 0.1 = 9.02 \text{ m/s. } x = 0 + 10 \times 0.1 = 1.0 \text{ m, } y = 0 + 9.02 \times 0.1 = 0.902 \text{ m.}$$

Exact: $y(0.1) = 10 \times 0.1 - \frac{1}{2} \times 9.8 \times 0.01 = 1 - 0.049 = 0.951$ m. The step error is 0.049 m; smaller Δt reduces it at the cost of more steps — exactly the kind of trade-off (accuracy vs computation) that makes fast computers valuable.

Subjective Questions with Answers

Q1. Name the three blocks of a computing system in the von Neumann-style model shown in the lecture, and explain why memory and storage are separated.

Answer

Computing unit (transforms data: arithmetic, logic, control), communication unit (moves data between blocks and to the outside world), and memory/storage unit (holds data and programs). Memory and storage are separated because no single technology is simultaneously fast, large, cheap and non-volatile: the *memory system* (registers, caches, DRAM) is fast but small and volatile; the *storage system* (SSD, disk) is large, cheap per bit and non-volatile but slow. Combining them gives the illusion of a memory that is both fast and large.

Q2. What is the “stored-program” idea, and why was it important?

Answer

Instructions are encoded as numbers and stored in the same memory as data. A new program can be loaded without rewiring the machine, programs can manipulate other programs (compilers, loaders, operating systems), and a single general-purpose machine can do any computable task. Earlier machines were often reconfigured physically for each task.

Q3. Explain the transformation hierarchy. Why do we need so many layers?

Answer

It is the sequence of abstraction levels from a computational problem down to physics: problem, algorithm, programming language, system software (compilers/OS), HW/SW interface (firmware, drivers, ISA), microarchitecture, logic and circuits, microelectronic devices, electrons, quantum mechanics. Each layer implements the layer above using the layer below and hides its internal details behind a contract. The gap between “solve this problem” and “move these electrons” is far too large for one step and one person; layers let specialists work at one level, reuse verified components, and change one layer (e.g. a new process technology) without redesigning everything.

Q4. Where does computer engineering sit in the hierarchy, and what does the “bridge” on the slide represent?

Answer

Between traditional computer science (problem, algorithms, languages) and traditional electrical engineering (electrons, devices, circuits): it spans system software, the HW/SW interface, microarchitecture and logic. The bridge represents its role of translating what software needs into what hardware can provide, and making trade-offs across that boundary.

Q5. Describe the physical hierarchy of a smartphone from device to transistor. Why can the total transistor count be so large?

Answer

Device $\rightarrow$ PCBs (~ 8) $\rightarrow$ ICs (8–16 per PCB) $\rightarrow$ modules (8–10 per IC) $\rightarrow$ standard cells (1K–10K per module) $\rightarrow$ gates (2–16 per cell) $\rightarrow$ transistors (2–8 per gate). The total is the *product* of the fan-outs at every level, so modest multipliers at six levels compound to millions or billions. Designers manage this by working at one level at a time.

Q6. Explain how a MOSFET acts as a switch. Why does holding its state cost almost no power?

Answer

In an nMOS transistor, two N^+ regions (source and drain) sit in P-type silicon, so normally no current flows between them (OFF). A positive gate voltage, applied across a thin oxide insulator, attracts electrons to the surface and forms a conducting N-channel between source and drain (ON). Because the gate is insulated by the oxide, essentially no steady current flows into it, so maintaining a voltage on it — holding the switch’s state — costs almost no power. Energy is spent mainly when charging or discharging during switching.

Q7. State Moore’s law. Is it a law of physics? What has happened to it in recent years?

Answer

The number of transistors on a dense integrated circuit doubles about every two years (Moore, 1965; revised from “every year” to “every two years” in 1975). It is an empirical observation and an industry roadmap target, not a physical law. Growth has slowed: power density limits ended rising clock speeds in the mid-2000s (prompting multi-core designs), and shrinking is approaching atomic dimensions and becoming very expensive. Density still improves, but more slowly and with help from 3D structures, stacking, and specialised accelerators.

Q8. Why does exponential growth appear as a straight line on the “shrinking chips” plot?

Answer

The plot uses a logarithmic y -axis. Taking logs of $N = N_0 2^{(t-t_0)/T_d}$ gives $\log N = \log N_0 + \frac{\log 2}{T_d}(t - t_0)$, which is linear in t with slope $\log 2 / T_d$. Constant doubling time means constant slope.

Q9. What makes a “good bit”? Evaluate electrical voltage against these criteria.

Answer

Small and inexpensive (we need billions), stable (reliable and repeatable), and easy and fast to manipulate (access, transform, combine, store, transmit). Voltage on a tiny wire or capacitor is extremely small and cheap per bit on a chip, switches in picoseconds to nanoseconds, and is directly produced and sensed by transistors. Its stability is its weakness: voltages are easily disturbed by noise and leakage, which is solved by the digital abstraction (restoring levels at every stage) and, for memories, by refreshing.

Q10. List the pros and cons of using voltage (rather than current, phase or frequency) to encode information inside a computer.

Answer

Pros: easy to generate and detect (transistor gates respond to voltage and transistor outputs produce voltage, so gates chain directly); decades of engineering knowledge; very low steady-state power (insulated gates draw almost no current). Cons: easily affected by environment (noise, crosstalk, supply variation); requires DC connectivity between driver and receiver; resistance and capacitance of wires slow down voltage changes (RC delay).

Q11. How much information is stored in one pixel encoded as an analog voltage between 0 V and 1 V? Justify.

Answer

In principle infinitely much, since the voltage is continuous. In practice it is limited by how finely we can distinguish voltages reliably and quickly: with a resolvable step ΔV , only about $N = V_{\text{range}}/\Delta V + 1$ levels are usable, giving $\log_2 N$ bits. Noise and component imperfections set ΔV , and the need for speed (nanoseconds per pixel) prevents averaging the noise away.

Q12. Explain how an image is turned into a single voltage waveform. What is the role of the sync pulse?

Answer

The 2-D image is scanned in a fixed raster order (left to right along a line, lines from top to bottom); each point's brightness becomes the voltage at the corresponding instant, producing a 1-D waveform in time. At the end of each line the signal drops to a sync level below black. Because no picture content is darker than black, the receiver can recognise sync unambiguously and use it to start each new line in step with the sender.

Q13. The COPY and INV blocks “work individually”. Why does a long chain of them still ruin the image?

Answer

Each real analog block has small gain and offset errors, limited bandwidth (which blurs edges) and noise. An analog block cannot tell which part of its input is signal and which is error, so it passes along all earlier errors and adds its own. Errors therefore accumulate: linearly ($N\epsilon$) in the worst case, as $\sqrt{N}\sigma$ for independent random noise. With enough stages the output is unusable, so “each block works” does not imply “the system works” for analog processing.

Q14. What is the digital abstraction? “Neither the real world is digital, nor the components” — then how can digital systems exist?

Answer

The digital abstraction interprets continuous voltages as just two values, 0 and 1, and requires every component to restore noisy inputs to clean, legal output levels. Physically everything remains analog: voltages vary continuously and components are imperfect. Digital is a design

discipline: as long as noise at each stage stays within the margin, each component snaps its output back to a valid level, so errors never accumulate and any number of components can be chained.

Q15. Compare the single-threshold scheme with the two-threshold (forbidden zone) scheme.

Answer

Single threshold: $V < V_{TH}$ is 0, $V \geq V_{TH}$ is 1. A signal just above or below V_{TH} can be flipped by arbitrarily small noise, and components with slightly different thresholds may disagree. Two thresholds: $V \leq V_L$ is 0, $V \geq V_H$ is 1, and senders must never output values in the forbidden zone between. Valid signals are therefore always separated from the other value by at least the zone width, which absorbs noise. (The slide's " $V_L > V > V_H$ " should read $V_L < V < V_H$.)

Q16. Define noise margins. Why must output thresholds be stricter than input thresholds?

Answer

$NM_H = V_{OH} - V_{IH}$ and $NM_L = V_{IL} - V_{OL}$: the amount of noise a signal can pick up between a sender and a receiver and still be read correctly. If senders were only required to produce $V \geq V_{IH}$ for a 1, any noise at all could push a borderline output into the forbidden zone. Requiring $V_{OH} > V_{IH}$ and $V_{OL} < V_{IL}$ leaves a safety gap. This is the static discipline.

Q17. Why does a digital gate need gain greater than one? Relate this to the shape of its transfer characteristic.

Answer

Restoration requires shrinking noise in the valid regions (flat parts of the transfer curve, slope magnitude < 1) and a steep transition between them. Going from nearly 0 output to nearly V_{DD} output across a narrow input range (the forbidden zone) requires slope magnitude > 1 , i.e. gain. A device with slope exactly 1 everywhere (like the ideal analog COPY) neither shrinks nor amplifies noise, so noise accumulates. Hence digital logic needs active devices such as transistors.

Q18. Why do computers use binary rather than, say, ten voltage levels per wire?

Answer

For a fixed voltage range, two levels give the largest spacing and the widest forbidden zone, hence the best noise immunity. Two-state devices (a transistor ON/OFF) are the simplest, smallest, fastest and lowest-power circuits. And binary maps directly onto Boolean logic (TRUE/FALSE), making design systematic. When more information is needed, more wires are used (n bits give 2^n values) rather than finer levels.

Q19. Prove that n bits can represent exactly 2^n distinct values.

Answer

Induction: 1 bit gives 2 values. If n bits give 2^n patterns, then each $(n + 1)$ -bit pattern is either 0 or 1 followed by an n -bit pattern, giving $2 \cdot 2^n = 2^{n+1}$ patterns. Hence 2^n for all $n \geq 1$. (Equivalently, n independent binary choices multiply to 2^n .)

Q20. Explain positional notation and show why every number from 0 to $b^n - 1$ has exactly one n -digit representation in base b .

Answer

The value is $V = \sum_{i=0}^{n-1} d_i b^i$ with $0 \leq d_i \leq b - 1$: each position has weight b^i . The maximum value of the lower i digits is $\sum_{j<i} (b - 1) b^j = b^i - 1$, one less than the weight of position i . So each position's contribution cannot be imitated by lower positions, and the b^n digit strings map one-to-one onto $0, \dots, b^n - 1$.

Q21. What were the contributions of Boole and Shannon, and why did digital electronics take about thirty years to follow the vacuum tube?

Answer

Boole (1854) showed that logic can be expressed as algebra on binary values (AND, OR, NOT with algebraic laws). Relays and the vacuum tube (1906) gave electrically controlled switches, but there was no systematic method for designing switching circuits. Shannon's 1937 MIT master's thesis showed that series switches implement AND, parallel switches implement OR, and switching networks obey Boolean algebra, so circuits could be designed and simplified mathematically. That missing theory, not missing hardware, was the delay.

Q22. The course emphasises understanding both hardware and software. Give one example each of how hardware knowledge improves software and how software knowledge improves hardware.

Answer

Hardware $\rightarrow$ software: knowing that memory accesses are slow compared with arithmetic and that caches favour nearby accesses, a programmer traverses arrays in memory order and gets large speed-ups. Software $\rightarrow$ hardware: knowing which instructions programs use most (simple loads, stores, adds, branches), architects make those fast and keep the instruction set simple — the reasoning behind RISC designs.

Chapter 11

Multiple-Choice Questions

Attempt all 40 first; answers with explanations follow in Section 11.2.

11.1 Questions

Orientation and computing blocks

1. The one word the opening slide highlights as the fundamental answer to “evolving and complex” systems is:
 - (a) Software
 - (b) Digital
 - (c) Silicon
 - (d) Parallel
2. Which textbook is the main text for CS3031?
 - (a) Harris & Harris, *Digital Design and Computer Architecture, ARM Edition*
 - (b) Patterson & Hennessy
 - (c) M. Morris Mano, *Digital Logic and Computer Design*
 - (d) Tanenbaum, *Structured Computer Organization*
3. Module 2 of the course designs combinational logic elements leading to:
 - (a) the CPU timing and control
 - (b) the ALU
 - (c) the instruction set
 - (d) memory management
4. In the 1946 Burks–Goldstein–von Neumann model shown in class, the memory/storage unit is subdivided into:
 - (a) cache and registers
 - (b) memory system and storage system
 - (c) ROM and RAM
 - (d) instruction memory and data memory
5. Which property distinguishes the storage system from the memory system?
 - (a) It is faster
 - (b) It is non-volatile and larger
 - (c) It sits inside the ALU
 - (d) It stores only instructions
6. The cannon picture on “Why do we need computation?” best illustrates:

- (a) that computers were invented for games
 - (b) a problem easy to state but laborious to solve numerically, such as firing tables
 - (c) Boolean algebra
 - (d) the von Neumann bottleneck
7. Without air resistance, a projectile's range is maximum at a launch angle of:
- (a) 30°
 - (b) 45°
 - (c) 60°
 - (d) 90°

Hierarchies

8. In the transformation hierarchy, which layer lies directly below "System Software (Compilers/OS)"?
- (a) Programming Languages
 - (b) Micro-Architecture
 - (c) HW/SW Interface (Firmware, Drivers)
 - (d) Logic
9. According to the slide, computer engineering primarily spans:
- (a) Computational problem to algorithms
 - (b) System software down to logic/circuits
 - (c) Electrons to quantum mechanics
 - (d) Only micro-electronic devices
10. Using the slide's ranges, the maximum number of transistors per IC is about:
- (a) 3.2×10^4
 - (b) 1.6×10^9
 - (c) 1.28×10^7
 - (d) 10^{12}
11. In an nMOS transistor, the conducting channel forms when:
- (a) the drain is grounded
 - (b) a sufficiently positive voltage is applied to the gate
 - (c) the oxide layer conducts
 - (d) current flows into the gate
12. The insulating layer between the gate and the silicon in a MOSFET is the:
- (a) N^+ region
 - (b) P-substrate
 - (c) oxide layer
 - (d) metallisation

13. “Lakhs” on the VLSI slide denotes about:

- (a) 10^3
- (b) 10^5
- (c) 10^7
- (d) 10^9

14. Why is a hierarchy essential for VLSI design?

- (a) It reduces transistor size
- (b) It lets designers handle billions of devices by reasoning about only a few units at each level
- (c) It removes the need for verification
- (d) It eliminates noise

Moore’s law and history

15. Moore’s law (as stated on the slide) says the number of transistors in a dense IC doubles about every:

- (a) year
- (b) 18 months
- (c) two years
- (d) five years

16. The company co-founded by Gordon Moore is:

- (a) Fairchild Camera and Instrument
- (b) Texas Instruments
- (c) Intel
- (d) AMD

17. With a two-year doubling time, the annual growth factor is:

- (a) 1.25
- (b) 1.414
- (c) 2
- (d) 4

18. Over 10 years at a two-year doubling time, the count grows by:

- (a) $10\times$
- (b) $20\times$
- (c) $32\times$
- (d) $100\times$

19. Exponential growth appears as a straight line when:

- (a) both axes are linear
- (b) the y -axis is logarithmic
- (c) the x -axis is logarithmic
- (d) data are averaged

20. The first transistor was built in:
- (a) 1906
 - (b) 1937
 - (c) 1947
 - (d) 1963
21. The first commercial CMOS chips (1974) appeared in:
- (a) mainframes
 - (b) digital watches
 - (c) telephone exchanges
 - (d) calculators for NASA
22. Halving the feature size (e.g. 90 nm $\rightarrow$ 45 nm) multiplies transistor density by roughly:
- (a) $\sqrt{2}$
 - (b) 2
 - (c) 4
 - (d) 8

Representing information

23. Which is *not* one of the slide's criteria for a good bit?
- (a) small, inexpensive
 - (b) stable
 - (c) ease and speed of manipulation
 - (d) continuous-valued
24. Which is listed on the slide as a *disadvantage* of voltage encoding?
- (a) very low power in steady state
 - (b) easy generation and detection
 - (c) resistive and capacitive effects slow things down
 - (d) engineering knowledge
25. With 0 V = black and 1 V = white, the information per analog pixel is fundamentally limited by:
- (a) the speed of light
 - (b) the number of pixels
 - (c) the ability to reliably and quickly distinguish voltages
 - (d) the display size
26. A 0–1 V signal with a 10 mV resolvable step gives how many usable levels?
- (a) 10
 - (b) 100
 - (c) 101
 - (d) 1024

27. In the raster-scan encoding, the sync pulse is placed below the black level so that:
- (a) the image appears darker
 - (b) it cannot be confused with image content
 - (c) it saves power
 - (d) colour can be encoded
28. A 640×480 image at 30 frames/s requires each pixel's voltage to be handled in about:
- (a) 1 ms
 - (b) 108 ns
 - (c) 10 ps
 - (d) 1 s

Computation, noise and the digital abstraction

29. The INV block of the slides maps an input voltage V (in $[0, 1]$ V) to:
- (a) V
 - (b) $-V$
 - (c) $1 - V$
 - (d) $1/V$
30. For N analog stages with independent random noise σ each, the total noise standard deviation is:
- (a) σ
 - (b) $\sqrt{N} \sigma$
 - (c) $N\sigma$
 - (d) $N^2\sigma$
31. The key property that prevents noise from accumulating in digital systems is that each element:
- (a) uses higher voltage
 - (b) restores noisy inputs to legal output values
 - (c) is faster
 - (d) is analog
32. Which statement matches the slide's banner?
- (a) The real world is digital but components are analog
 - (b) Components are digital but the world is analog
 - (c) Neither the real world nor the components are digital
 - (d) Both are digital
33. The main weakness of a single-threshold scheme is:
- (a) it needs two supplies
 - (b) signals near the threshold can be flipped by tiny noise
 - (c) it cannot represent 1
 - (d) it uses too much power

34. In the two-threshold scheme, the forbidden zone correctly written is:
- (a) $V_L > V > V_H$
 - (b) $V < V_L$
 - (c) $V_L < V < V_H$
 - (d) $V > V_H$
35. Given $V_{OH} = 2.7$, $V_{IH} = 2.0$, $V_{IL} = 0.8$, $V_{OL} = 0.4$ (V), the low noise margin is:
- (a) 0.4 V
 - (b) 0.7 V
 - (c) 1.2 V
 - (d) 2.3 V
36. In the slide's binary system (3 V / 2.5 V / 0.5 V / 0 V), noise of 0.6 V on a "1":
- (a) is always harmless
 - (b) may push the signal into the forbidden zone
 - (c) turns it into a clean "0"
 - (d) increases the margin

Bits, numbers and Boolean logic

37. How many distinct values can 5 bits represent?
- (a) 5
 - (b) 10
 - (c) 25
 - (d) 32
38. 1011_2 equals:
- (a) 9
 - (b) 10
 - (c) 11
 - (d) 13
39. Which operation outputs 1 only when both inputs are 1?
- (a) OR
 - (b) AND
 - (c) NOT
 - (d) XOR
40. Shannon's 1937 thesis showed that:
- (a) transistors can be made of silicon
 - (b) switching circuits can be analysed with Boolean algebra
 - (c) transistor counts double every two years
 - (d) programs can be stored in memory

11.2 Answers and Explanations

1 b	2 c	3 b	4 b	5 b	6 b	7 b	8 c	9 b	10 c
11 b	12 c	13 b	14 b	15 c	16 c	17 b	18 c	19 b	20 c
21 b	22 c	23 d	24 c	25 c	26 c	27 b	28 b	29 c	30 b
31 b	32 c	33 b	34 c	35 a	36 b	37 d	38 c	39 b	40 b

1. **(b)** The slide flashes DIGITAL: the digital abstraction is what stays fundamental across generations.
2. **(c)** Mano is the main text; Harris & Harris is the advanced reference.
3. **(b)** Module 2: combinational logic → ALU. Module 3 covers timing and control.
4. **(b)** The slide splits the memory/storage unit into a memory system and a storage system.
5. **(b)** Storage (SSD, disk) is non-volatile and large but slower.
6. **(b)** Firing tables motivated early electronic computers; with drag, trajectories need step-by-step numerical computation.
7. **(b)** $R = v_0^2 \sin 2\theta / g$ is largest when $\sin 2\theta = 1$, i.e. $\theta = 45^\circ$.
8. **(c)** Order: ..., System Software, HW/SW Interface, Micro-Architecture, ...
9. **(b)** The red “bridge” on the slide runs from system software to logic/circuits.
10. **(c)** $10 \times 10^4 \times 16 \times 8 = 1.28 \times 10^7$ (modules $\times$ cells $\times$ gates $\times$ transistors). (a) is the per-IC minimum; (b) is the per-device maximum.
11. **(b)** A positive gate voltage attracts electrons under the oxide, forming an N-channel between source and drain.
12. **(c)** The oxide (SiO_2) insulates the gate; that is why the gate draws almost no current.
13. **(b)** 1 lakh = 10^5 .
14. **(b)** Hierarchy lets you reason about ~ 10 units at a time; the number of levels grows only logarithmically with total size.
15. **(c)** Two years (Moore’s 1975 revision; his 1965 paper said roughly every year).
16. **(c)** Intel (1968, with Robert Noyce). Fairchild was his employer when writing the 1965 paper.
17. **(b)** $2^{1/2} \approx 1.414$, i.e. about 41% per year.
18. **(c)** $10/2 = 5$ doublings: $2^5 = 32$.
19. **(b)** $\log N$ is linear in t for exponential growth.
20. **(c)** 1947, Bell Labs (Bardeen, Brattain, Shockley; Nobel Prize 1956).
21. **(b)** Digital watches, where CMOS’s very low static power was decisive.
22. **(c)** Density $\propto 1/L^2$; $(90/45)^2 = 4$.
23. **(d)** Good bits are small/cheap, stable, and fast to manipulate. Continuous-valued signals are the opposite of what the digital abstraction wants.
24. **(c)** RC effects are listed as a con; the other three are pros.
25. **(c)** Noise and the need for speed limit how finely voltages can be told apart.
26. **(c)** $\lfloor 1/0.01 \rfloor + 1 = 101$ (fence-post counting includes both ends).

27. **(b)** Nothing in the image is darker than black, so a below-black pulse is unambiguous.
28. **(b)** $1/(640 \times 480 \times 30) \approx 108.5 \text{ ns}$.
29. **(c)** INV produces the complement (negative image): $1 - V$.
30. **(b)** Independent variances add: $N\sigma^2$; standard deviation $\sqrt{N}\sigma$.
31. **(b)** Restoration at every stage wipes out noise before it can accumulate.
32. **(c)** Physics and components are analog; digital is an interpretation and design discipline.
33. **(b)** Near V_{TH} , arbitrarily small noise changes the interpretation.
34. **(c)** The slide's " $V_L > V > V_H$ " is a typo; since $V_L < V_H$ the zone is $V_L < V < V_H$.
35. **(a)** $NM_L = V_{IL} - V_{OL} = 0.8 - 0.4 = 0.4 \text{ V}$. ($NM_H = 0.7 \text{ V}$.)
36. **(b)** $3 - 0.6 = 2.4 \text{ V} < 2.5 \text{ V}$: the noise exceeds the 0.5 V margin, so the signal may land in the forbidden zone.
37. **(d)** $2^5 = 32$.
38. **(c)** $8 + 0 + 2 + 1 = 11$.
39. **(b)** AND is 1 only when all inputs are 1.
40. **(b)** Series switches = AND, parallel = OR: switching networks obey Boolean algebra.

Appendix A

Formula Sheet, Glossary and Quick Revision

A.1 Formula sheet

Quantity	Formula	Units / notes
Patterns from n bits	$N = 2^n$	count; values $0 \dots 2^n - 1$
Bits needed for N items	$n = \lceil \log_2 N \rceil$	bits
Information in one of N levels	$I = \log_2 N$	bits
Analog levels	$N = \lfloor V_{\text{range}} / \Delta V \rfloor + 1$	V/V, dimensionless
Positional value	$V = \sum_{i=0}^{n-1} d_i b^i$	$0 \leq d_i \leq b - 1$
Max value, n digits base b	$b^n - 1$	from geometric series
Hierarchy count	$N = \prod_i m_i$	count
Moore's law	$N(t) = N_0 2^{(t-t_0)/T_d}$	T_d in years
Annual growth factor	$2^{1/T_d}$	1.414 for $T_d = 2$
Doubling time from data	$T_d = \frac{(t_2 - t_1) \ln 2}{\ln(N_2/N_1)}$	years
Time for factor k	$\Delta t = T_d \log_2 k$	years
Density vs feature size	$D_2/D_1 = (L_1/L_2)^2$	L in m
Raster pixel rate	$R = WHF, t_{\text{pix}} = 1/R$	pixels/s, s
Analog error (worst)	$ E_N \leq N\varepsilon$	V
Analog error (random)	$\sigma_{E_N} = \sqrt{N}\sigma$	V
Gain error	$(1 + \delta)^N \approx 1 + N\delta$	small $N\delta$
Noise margins	$NM_H = V_{OH} - V_{IH},$ $NM_L = V_{IL} - V_{OL}$	V
Multi-level tolerance	$v_{n,\text{max}} = \frac{V_{\text{range}}}{2(L-1)}$	V
RC charging	$V(t) = V_{DD}(1 - e^{-t/RC})$	$\tau = RC$ in s
RC delays	$t_{50} = RC \ln 2, t_{90} = RC \ln 10$	s
Projectile range	$R = v_0^2 \sin 2\theta / g$	m
Flight time, height	$t = 2v_0 \sin \theta / g, H = \frac{v_0^2 \sin^2 \theta}{2g}$	s, m

A.2 Glossary

Abstraction A simplified view of a component that exposes what it does and hides how.

ALU Arithmetic and Logic Unit: performs arithmetic and logical operations; built in Module 2.

Analog Represented by a continuously variable physical quantity.

Binary Base-2; exactly two values, 0 and 1.

Bit Binary digit: the basic unit of information, one choice between two values.

Boolean algebra Algebra on $\{0, 1\}$ with AND, OR, NOT (Boole, 1854).

Cell (standard cell) Pre-designed layout of a small logic function in a chip library.

CMOS Complementary MOS: pairs of nMOS and pMOS transistors; almost no static power.

Digital abstraction Interpreting analog voltages as discrete values and restoring them at each stage so noise does not accumulate.

Forbidden zone The voltage range between V_L and V_H that valid outputs must never occupy.

IC Integrated circuit: a silicon die with many transistors, packaged as a chip.

ISA Instruction Set Architecture: the HW/SW contract; Module 4.

Microarchitecture How an ISA is implemented in hardware: datapath, pipelines, caches.

Moore's law Transistor counts double roughly every two years (empirical).

MOSFET Metal–oxide–semiconductor field-effect transistor: a voltage-controlled switch.

Noise Unwanted disturbance of a signal from the environment or imperfect components.

Noise margin How much noise a digital signal can tolerate and still be read correctly.

PCB Printed circuit board carrying and connecting chips.

Raster order Line-by-line, left-to-right, top-to-bottom scanning of a 2-D image.

Restoration A digital element outputting a clean legal level from a slightly noisy input.

Static discipline Rule that outputs meet stricter thresholds (V_{OL} , V_{OH}) than inputs require (V_{IL} , V_{IH}).

Stored program Instructions stored in memory as data (von Neumann).

Sync pulse Below-black signal marking the start of a new scan line.

Transformation hierarchy The layers from computational problem down to quantum mechanics.

VLSI Very Large-Scale Integration: chips with very many (now billions of) transistors.

Volatile Loses its contents when power is removed.

A.3 One-page revision checklist

Tick each when you can explain it without notes:

- ☐ The three questions of the course and why the answer is “digital”.
- ☐ The four modules and what each builds.
- ☐ The three computing blocks; memory vs storage; stored-program idea.
- ☐ Why the cannon: problems $\rightarrow$ computation $\rightarrow$ insight.
- ☐ All ten layers of the transformation hierarchy and where CE sits.
- ☐ Device $\rightarrow$ PCB $\rightarrow$ IC $\rightarrow$ module $\rightarrow$ cell $\rightarrow$ gate $\rightarrow$ transistor, with multiplying counts.

- ☐ How a MOSFET switches; why holding state costs almost no power.
- ☐ Moore's law formula, T_d from two data points, log-plot straight line; Intel.
- ☐ Milestones: 1854, 1906, 1937, 1946, 1947, 1954, 1963, 1965, 1974, 2020.
- ☐ What makes a good bit; voltage pros and cons.
- ☐ Analog pixel information: infinite in principle, $\log_2 N$ in practice.
- ☐ Raster scan and sync pulses.
- ☐ COPY/INV functions; why chains of analog blocks blur.
- ☐ $N\varepsilon$ vs $\sqrt{N}\sigma$ accumulation.
- ☐ The digital abstraction and the banner statement.
- ☐ Single threshold vs forbidden zone (and the slide typo).
- ☐ Noise margins and the static discipline.
- ☐ 2^n , $\lceil \log_2 N \rceil$, positional notation, conversions.
- ☐ AND/OR/NOT truth tables; Boole; Shannon's series/parallel insight.